Continual Low-Rank Adaptation via Cumulative Unified Optimization

Yue Lu, Shizhou Zhang✉, De Cheng✉, Yinghui Xing, *Member, IEEE,* Guoqiang Liang, Nannan Wang, *Senior Member, IEEE,* Peng Wang, *Member, IEEE,* Yanning Zhang, *Fellow, IEEE*

***Abstract*—Low-Rank Adaptation (LoRA) enables foundation models to efficiently adapt to downstream tasks with minimal trainable parameters, yet it lacks the capability of continual learning (CL) from non-stationary data streams. While existing CL methods attempt to address this by storing and retrieving task-specific parameter modules, they suffer from non-trivial challenges of task identification and inconsistency in input-output feature mappings, resulting in suboptimal feature representations. Instead of keeping multiple task-specific parameter modules, we propose a novel continual LoRA mechanism that parameterizes all task-specific adaptations with a single adaptation module, naturally obviating the task identification and mapping inconsistency issues. To this end, we reformulate LoRA-based CL as a consistent feature mapping problem that mimics the behavior of the joint-training upper bound, wherein a *unified adaptation parameter matrix* is learned to simultaneously capture the input-output relationships established by all task-specific LoRAs. By solving this formulation, our approach facilitates approximate unified optimization over sequential tasks via progressively updating *cumulative feature statistics*, achieving efficient knowledge retention and renewal. Notably, the proposed arithmetic training mechanism can be applied both to the backbone of the foundation model for parameter-efficient fine-tuning and the classifier optimization, thereby enabling continual learning within an integrated architecture that closely approximates the joint learning paradigm. Extensive experiments across conventional and long-sequence class-incremental benchmarks demonstrate that our method achieves new state-of-the-art performance. Our code is available at https://github.com/zugexiaodui/ContLoRAvCUO.**

***Index Terms*—Continual learning, class-incremental learning, low-rank adaptation, catastrophic forgetting**

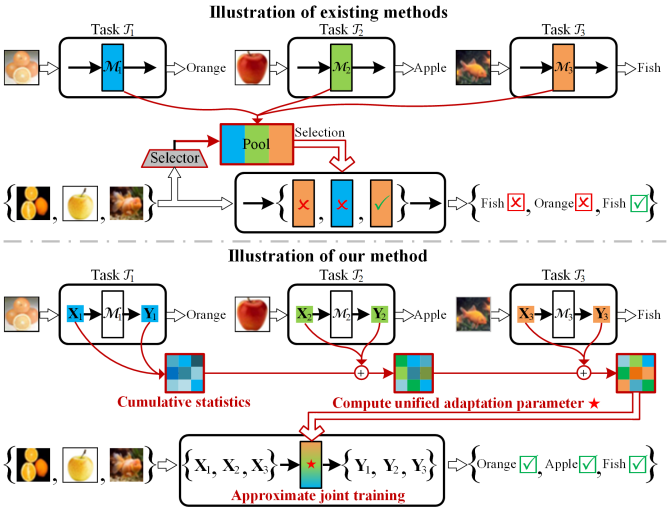


Fig. 1. Illustration of existing and proposed methods. Existing methods typically preserve learned knowledge by storing task-specific parameter modules ($\mathcal{M}_{\{1,2,3\}}$) and selecting the most relevant one for prediction. Our unified optimization method retains knowledge by maintaining cumulative statistics of input-output features. The statistics are subsequently used to derive the unified adaptation parameter module that aims to approximate the behavior of joint training.

I. Introduction

FOUNDATION models implemented by Transformers [1], [2] have achieved remarkable generalization across diverse domains [3]–[5], yet full fine-tuning in downstream tasks remains computationally and memory intensive. Low-Rank Adaptation (LoRA) [6] has recently emerged as a parameter-efficient approach for fine-tuning foundation models. By introducing only a small number of trainable low-rank matrices, LoRA achieves performance comparable to or surpassing full fine-tuning parameters [6]. Owing to its efficiency and modular design, LoRA is widely employed in diverse applications, *e.g.*, QLoRA [7] and Alpaca-LoRA [8] for instruction-tuning large-scale language and vision models. Nevertheless, real-world applications such as autonomous driving and robotic perception demand models capable of *continual learning* (CL) from non-stationary data streams [9]–[12]. In such dynamic environments, LoRA suffers from *catastrophic forgetting* [13]–[15], as newly adapted parameters disrupt previously learned representations. As a result, it fails to *preserve acquired knowledge* (*i.e.*, *stability*) when *adapting to new tasks* (*i.e.*, *plasticity*).

In the field of continual learning with foundation models, researchers have explored various parameter-efficient fine-tuning strategies, including Prompt tuning [16], Adapter [17], and LoRA [6]. Prompt-based CL methods [18]–[24] typically train a set of prompts (or prefixes [25]) for each task and store them

This work was supported in part by the National Natural Science Foundation of China (NSFC) under Grant 62576282, 62576262 and 62201467; in part by the Young Talent Fund of Xi'an Association for Science and Technology under Grant 959202313088, in part by Innovation Capability Support Program of Shaanxi (Program No. 2024ZC-KJXX-043) and in part by the Natural Science Basic Research Program of Shaanxi Province (No. 2024JC-DXWT-07).

Yue Lu, Shizhou Zhang, Yinghui Xing, Guoqiang Liang, Peng Wang and Yanning Zhang are with School of Computer Science, Northwestern Polytechnical University, Xi'an, China (email: zgxd@mail.nwpu.edu.cn, szzhang@nwpu.edu.cn, xyh_7491@nwpu.edu.cn, gqliang@nwpu.edu.cn, peng.wang@nwpu.edu.cn, ynzhang@nwpu.edu.cn). De Cheng and Nannan Wang are with School of Telecommunications Engineering, Xidian University, Xi'an, China (email: dcheng@xidian.edu.cn, nnwang@xidian.edu.cn). ✉Corresponding authors: Shizhou Zhang (szzhang@nwpu.edu.cn), De Cheng (dcheng@xidian.edu.cn).

0000–0000/00$00.00 © 2021 IEEE

in a prompt pool, from which appropriate prompts are retrieved during inference. Adapter-based approaches [26]–[30] often follow a branch-expansion design, where each task is assigned a specific feature space [26] or integrates features from multiple adapters [29]. Benefiting from the flexibility and simplicity, LoRA has been increasingly adopted in CL [31]–[38]. Approaches based on LoRA incorporate techniques including slow parameter updates [32], orthogonality constraints [34] and parameter arithmetic [35]. Most existing methods encode knowledge from all tasks by storing task-specific parameter modules (as illustrated in the upper part of Fig. 1). During inference, they either select the most relevant task-specific parameter module (*e.g.*, expert prompts in DualPrompt [19], task-specific adapters in CL-LoRA [38]), or select multiple related modules and fuse them through addition or magnitude-based merge (*e.g.*, MoE-LoRA in BCL [36], task vector fusion in MagMax [39]). Through a further comparison between the task-specific training-based CL and the *joint training* (the *upper bound* of CL) paradigms in terms of parameter-feature interactions, we find that multiple task-specific LoRAs in CL hinder the mapping of input features to their corresponding fine-tuned outputs via *a single deterministic* LoRA. This fundamental discrepancy reveals two limitations in those methods: (1) the model performance depends on accurately identifying the corresponding task-specific parameter module during inference, which is non-trivial and prone to prediction errors; (2) the isolation of task-specific parameter modules leads to inconsistent input-output mappings, thereby yielding suboptimal feature representations.

In this work, we propose a novel continual LoRA mechanism that parameterizes all task-specific adaptations with a single unified adaptation module, which aims to approximate the behavior of joint training and naturally avoids task identification and input-output mapping inconsistency. This mechanism facilitates a unified optimization over sequential tasks through maintaining and updating *cumulative feature statistics* instead of multiple task-specific parameter modules, as illustrated in the lower part of Fig. 1. Specifically, we reformulate LoRA-based CL as a consistent input-output feature mapping problem across tasks, so as to obtain a *unified adaptation parameter matrix* that can simultaneously fit the input-output feature relationships established by all trained task-specific LoRAs within a backbone block. This formulation leads to a convex least-squares objective, which admits a closed-form approximately joint training *solution* (*i.e.*, *the unified adaptation parameter matrix*) via the normal equations. The solution is reformulated into a *progressive form* based on fixed-dimensional *cumulative statistics* of LoRA's input and output features. This enables the cumulative statistics to be progressively updated using only the input-output features of LoRAs trained on the new task, thereby achieving efficient knowledge preservation and renewal. Ultimately, the unified adaptation parameter matrix derived from the cumulative statistics establishes consistent input-output feature mappings across all tasks.

Beyond the LoRA modules in the backbone, the classifier also requires a similar feature-mapping mechanism to enable unified optimization and mitigate cross-task confusion [40]–[42]. However, directly applying the approach tailored for LoRA to the classifier is hindered by two main challenges: (1) the output dimensionality grows with the number of classes; (2) when the input features and classifier weights originate from different tasks, the ground-truth output logits are unavailable. To address these, we first introduce a global classifier and redefine its output space, enabling us to reformulate its CL objective within the same feature mapping framework consistent with LoRA. Through further analysis, we overcome the obstacles by incorporating additional cumulative feature statistics with specifically derived progressive updating rules, and by approximating cross-task ground-truth logits through scaled predicted logits. Consequently, both the backbone equipped with LoRAs and the classifier can be optimized toward approximate joint training within an integrated framework.

Our approach exhibits advantages in both analytical soundness and empirical performance. We derive two properties of our method in CL: (1) The forgetting magnitude is bounded by the performance gain on new tasks and does not exceed half of it, thereby ensuring that adaptation to new tasks does not incur severe forgetting of previous tasks. (2) With the integration of new tasks, the progressive updates of the solution ensure a monotonically decreasing trajectory of the overall optimization objective, with each reduction measured by the Mahalanobis norm under the uncentered covariance of input features. In experimental evaluations, our method substantially outperforms existing approaches, with the advantage increasing in longer task sequences. We summarize our contributions as follows:

- We establish a unified optimization framework for LoRA-based continual learning under a consistent feature mapping mechanism, which enables knowledge preservation through cumulative statistics of features, and achieves approximate joint training of LoRA via progressive updates to these cumulative statistics.
- We extend the feature mapping principle to classifiers within an integrated framework by introducing additional cumulative statistics with specifically derived progressive updating rules, and by approximating cross-task ground-truth logits through scaled predicted logits, thus effectively mitigating cross-task confusion.
- Our method achieves new state-of-the-art performance across multiple class-incremental learning benchmarks and demonstrates substantial improvements over existing approaches, particularly in challenging long-sequence scenarios.

II. Related Work

Benefiting from the strong generalization ability of pre-trained foundation models, recent studies have leveraged parameter-efficient tuning techniques to explore new paradigms for continual learning. Existing approaches can be broadly divided into the following categories according to their anti-forgetting mechanisms:

1) Prompt pool-based methods [18]–[24], [43]–[56] represent the mainstream approach. For example, L2P [18] and DualPrompt [19] encode task knowledge into shared or task-specific prompt pool during training, and retrieve the most

relevant prompt for inference. CODA-Prompt [20] further employs a learnable soft-attention mechanism to combine all prompts, which enables end-to-end training. CPrompt [44] uniformly samples prompts from the pool during training, enabling the model to maintain competitive inference accuracy even when an incorrect prompt is selected. VPT-CPG [22] trains a mixture-of-experts model and utilizes the router to select and aggregate multiple relevant expert prompts. However, determining the appropriate task-specific parameters is non-trivial, and the dependence on a two-stage inference pipeline significantly increases the computational cost in practice.

2) Regularization-based methods [29], [31], [33], [34], [49], [53], [57] aim to preserve stability by constraining parameter updates during training. Typical regularization strategies mainly include knowledge distillation [29], [53], [58] and orthogonal subspace projection [31], [33], [34], [49], [57]. The orthogonal projection technique has recently attracted increasing attention in parameter-efficient continual learning due to its theoretical guarantees. PGP [57] and VPT-NSP^2 [49] analyze the stability objective and derive orthogonality conditions specifically for prompt tuning, which are implemented through GPM [59] and null-space projection [60]. InfLoRA [31] projects the gradients of new LoRA branches onto subspaces orthogonal to the gradient space of previous tasks, thereby alleviating interference across tasks. BiLoRA [34] further develops a bilinear reformulation that can achieve almost orthogonality as the parameter space expands. While these methods provide reliable stability, the subspace projection inevitably weakens plasticity, hindering a well-balanced stability-plasticity trade-off.

3) Task arithmetic-based approaches [30], [35], [39], [61]–[65] aim to integrate knowledge across tasks through algebraic manipulation at the parameter level. A common strategy is to merge task-specific parameters learned from observed tasks. For instance, LAE [61] employs an exponential moving average (EMA) to combine new-task parameters with those from previous tasks, and assigns smaller weights to the new parameters to maintain stability. TMC [62] proposes that multiple models can be fused into a single model via linear combination under the assumption of small parameter updates. MagMax [39] replaces conventional summation in parameter fusion with maximum-magnitude selection, thereby retaining the parameters critical for each task. Although these approaches can mitigate the task identification issue through parameter-level fusion, they still lack an explicit mechanism to guarantee that input features are consistently mapped to their corresponding output features. Instead of directly merging parameters, several approaches (CNN-based: [63]–[65], Transformer-based: [30]) address incremental learning of the classification head by applying linear regression to features and classification scores to derive fused classifier weights. These methods fall under an analytic form of task arithmetic, and our proposed method can also be viewed in this light. Nonetheless, our approach differs from these methods [30], [63]–[65] in three critical aspects:

(1) At the framework level, existing methods focus solely on incremental representation recognition in the classification head while neglecting representation learning in the backbone (where the backbone is typically frozen [63]–[65] or updated only through momentum [30]), which limits their ability in continual learning. In contrast, our method unifies task arithmetic for representation learning and recognition within one framework, enabling adapters to learn continually with improved plasticity-stability balance and reduced cross-task confusion. (2) At the optimization solution level, these methods derive the weights for the classifier under restrictive assumptions, resulting in suboptimal solutions. We propose a more generalized solution for incremental representation recognition through a comprehensive analysis. As a result, existing solutions can be regarded as a special case of our derived solution. Experiments further demonstrate that our generalized solution substantially outperforms prior works on both conventional and long-sequence CL benchmarks. (3) At the representation learning level, these methods rely on high-dimensional random projections to refine representations due to the lack of analysis on representation learning. Benefiting from the proposed optimization framework with an integrated design of backbone and classifier, our method eliminates the need for such projections, thereby reducing computational cost and enhancing the generalization of learned representations.

Connection to recursive least squares: The progressive derivation in our method is also related in form to recursive least squares (RLS) and its kernel extension KRLS [66], which recursively update least-squares solutions from streaming data. This resemblance mainly arises from the linear structure of LoRA. However, our motivation and learning objective are fundamentally different from conventional RLS/KRLS. Such methods are typically developed for online regression or system identification with a fixed output space, where a forgetting factor is often introduced to discount outdated observations under non-stationary signals. In contrast, our method targets class-incremental continual learning, where knowledge from previous tasks should be retained rather than discounted, and the output space progressively expands as new classes arrive. As a result, our progressive derivation with cumulative statistics is derived from approximating joint training over all seen tasks. By serving as an explicit memory of previous tasks, the accumulated statistics continuously retain historical knowledge and enable the unified solution to balance stability and plasticity in continual learning.

III. Preliminaries

A. Continual Learning

In continual learning, a model is trained sequentially on a stream of tasks $\mathcal{T}_1, \mathcal{T}_2, \ldots, \mathcal{T}_T$, where T denotes the total number of tasks. For the t-th task $\mathcal{T}_t$, the corresponding training set is denoted as $\mathcal{D}_t = \{(x_t^{(n)}, y_t^{(n)})\}_{n=1}^{|\mathcal{T}_t|}$, where $x_t^{(n)}$ is the n-th input instance, $y_t^{(n)}$ is its associated label, and $|\mathcal{T}_t|$ denotes the number of samples in task $\mathcal{T}_t$. We consider the widely adopted class-incremental learning (CIL) setting [67], where the label space of each task is disjoint, *i.e.*, $\mathcal{Y}_t \cap \mathcal{Y}_{t'} = \emptyset$ for all $t \neq t'$. After completing the training on $\mathcal{T}_t$, the dataset $\mathcal{D}_t$ is discarded and cannot be revisited during subsequent training. At inference time, the model is evaluated on test samples from the union of all previously seen classes, without

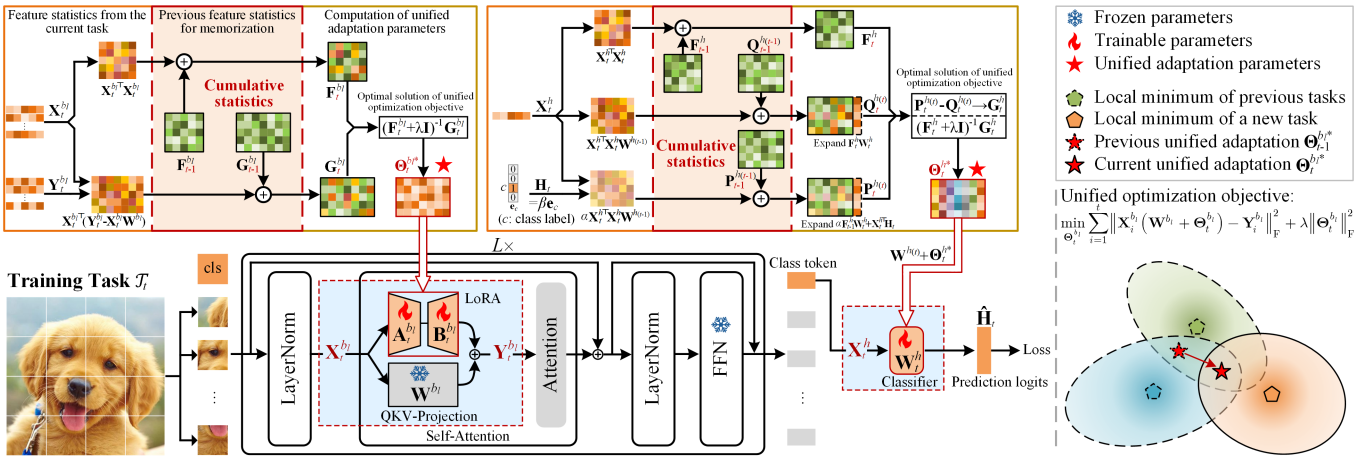


Fig. 2. Overview of the proposed framework. In the task $\mathcal{T}_t$, only the LoRA parameters $\mathbf{A}_t^{b_l}, \mathbf{B}_t^{b_l}$ $(l \in \{1, 2, \cdots, L\})$ and the task-specific classifier $\mathbf{W}_t^h$ are trained. After training, the unified adaptation parameter matrices $\mathbf{\Theta}_t^{b_l *}$ and $\mathbf{\Theta}_t^{h*}$ are computed for LoRAs and classifiers via updated cumulative feature statistics, which are then employed for prediction during inference. As depicted on the right, the star denotes the solution of unified adaptation, which achieves a low loss across multiple tasks. When a new task arrives, the solution is progressively updated toward a new optimum over all observed tasks.

access to the task identifier. In our notation, t denotes the index of the current task, and $i \in \{1, 2, \ldots, t\}$ refers to the index of an observed task among the first t tasks.

B. Low-Rank Adaptation

Low-Rank Adaptation (LoRA) [6] is a parameter-efficient fine-tuning technique for adapting large-scale pre-trained foundation models to downstream tasks. Given a pre-trained weight matrix $\mathbf{W} \in \mathbb{R}^{D\times E}$ in a layer of the backbone, LoRA introduces two trainable low-rank matrices $\mathbf{A} \in \mathbb{R}^{D\times r}$ and $\mathbf{B} \in \mathbb{R}^{r\times E}$, where D and E denote the input and output dimensions of the weight matrix, respectively, and $r \ll \min(D, E)$ is the rank of the decomposition. The adapted weight is then expressed as: $\mathbf{W}' = \mathbf{W} + \mathbf{AB}$. This formulation reduces the number of trainable parameters from $\mathcal{O}(DE)$ to $\mathcal{O}(r(D+E))$, while still allowing sufficient flexibility to capture task-specific adaptations. By keeping the original weights $\mathbf{W}$ frozen and only training the low-rank components, LoRA enables efficient fine-tuning with minimal memory and computation overheads.

IV. Method

A. Overall Framework

The framework of our method is illustrated in Fig. 2. We adopt the Vision Transformer (ViT) [2] as the backbone. The input image is divided into M non-overlapping patches, each projected into a D-dimensional token. A class token is prepended to the token sequence, and positional embeddings are added to preserve spatial information. The resulting sequence of length $N = M+1$ is then processed by L Transformer blocks, where each block b_l $(l = 1, \ldots, L)$ consists of LayerNorm [68] layers, a multi-head self-attention module, and a feed-forward network (FFN). The output sequence of the final block is denoted as $\mathbf{Y}^{b_L}$, from which the class token $\mathbf{y}_{\text{cls}}^{b_L} \in \mathbb{R}^D$ is extracted and fed into a task-specific classifier. The logits output by the classifier are used to compute the cross-entropy loss ℓ, which is minimized via gradient descent to train the model. To achieve efficient fine-tuning, we insert LoRA modules into the query, key, and value projection (QKV-projection) layer preceding the attention operation [31]–[33]. During training the t-th task $\mathcal{T}_t$, each block b_l is equipped with trainable LoRA parameters $\mathbf{A}_t^{b_l}, \mathbf{B}_t^{b_l}$, while the pre-trained QKV-projection weights $\mathbf{W}^{b_l}$ remain frozen. In addition, the task-specific classifier $\mathbf{W}_t^h \in \mathbb{R}^{D\times C_t}$ is trained, where C_t denotes the number of classes in $\mathcal{T}_t$. To sum up, only the LoRA parameters and the current classifier are trained for each task.

After completing the training of $\mathcal{T}_t$, we compute two types of unified adaptation parameter matrices: $\mathbf{\Theta}_t^{b_l *}$ and $\mathbf{\Theta}_t^{h*}$, which integrate the knowledge from all previously observed tasks within the backbone and classifier, respectively. These matrices are updated progressively via closed-form solutions based on cumulative statistics of features, ensuring efficient knowledge preservation without revisiting previous datasets. At inference time, we replace LoRA modules with the unified adaptation matrix $\mathbf{\Theta}_t^{b_l *}$. The task-specific classifiers $\{\mathbf{W}_1^h, \ldots, \mathbf{W}_t^h\}$ are concatenated to construct a global classifier $\mathbf{W}^{h_{(t)}}$, whose weights are further refined by adding the unified adaptation matrix $\mathbf{\Theta}_t^{h*}$. This design enables the model to make predictions over all previously learned classes using a single backbone and a global classifier, avoiding the need to infer task identifiers.

In the following subsections, we formulate the feature mapping scheme, and subsequently derive progressive solutions for the unified adaptation parameters of the backbone and the classifier, respectively.

B. Joint Training Paradigm

A model is required to adapt to a sequential stream of available tasks $\mathcal{T}_1, \mathcal{T}_2, \cdots, \mathcal{T}_t$ in continual learning. The performance upper bound (referred to as the *joint upper bound*) is achieved by training the entire model $f(\cdot)$ jointly on all datasets $\mathcal{D}_1 \cup \mathcal{D}_2 \cup \cdots \cup \mathcal{D}_t$ through minimizing the overall classification loss:

$$\min_{\tilde{\mathbf{\Omega}}} \mathbb{E}_{(x,y)\sim\mathcal{D}_1\cup\mathcal{D}_2\cdots\cup\mathcal{D}_t} \ell\big(f(x;\tilde{\mathbf{\Omega}}), y\big), \tag{1}$$

where $\tilde{\mathbf{\Omega}}$ denotes the parameters of the model, consisting of the frozen pre-trained parameters in the ViT backbone, the trainable LoRA parameters $\tilde{\mathbf{A}}^{b_l}, \tilde{\mathbf{B}}^{b_l}$ for the l-th block ($l \in \{1, 2, \cdots, L\}$), and the classifier weights $\tilde{\mathbf{W}}^{h_{(t)}}$. In this paper, we use the tilde notation '˜' to indicate parameters or features obtained under the joint training paradigm. The superscript $h_{(t)}$ denotes the global classifier of the first t tasks. The parameters $\tilde{\mathbf{A}}^{b_l}, \tilde{\mathbf{B}}^{b_l}$ and $\tilde{\mathbf{W}}^{h_{(t)}}$ are trained from all original inputs x and labels y via gradient descent, allowing the model to generalize well to different tasks.

C. Analysis of Feature Mapping in Each Block

Next, we delve into each ViT block to analyze the relationship between the trained LoRAs and the features. For the QKV-projection layer in the l-th block, we denote the input features from $\mathcal{D}_i$ as $\tilde{\mathbf{X}}_i^{b_l}$ and the output features as $\tilde{\mathbf{Y}}_i^{b_l}$. The relationship between the parameters $\tilde{\mathbf{A}}^{b_l}, \tilde{\mathbf{B}}^{b_l}, \mathbf{W}^{b_l}$ and the corresponding input-output features can be expressed as:

$$\begin{cases} \tilde{\mathbf{X}}_1^{b_l}(\mathbf{W}^{b_l} + \tilde{\mathbf{A}}^{b_l}\tilde{\mathbf{B}}^{b_l}) = \tilde{\mathbf{Y}}_1^{b_l}, \\ \tilde{\mathbf{X}}_2^{b_l}(\mathbf{W}^{b_l} + \tilde{\mathbf{A}}^{b_l}\tilde{\mathbf{B}}^{b_l}) = \tilde{\mathbf{Y}}_2^{b_l}, \\ \qquad\qquad\qquad \vdots \\ \tilde{\mathbf{X}}_t^{b_l}(\mathbf{W}^{b_l} + \tilde{\mathbf{A}}^{b_l}\tilde{\mathbf{B}}^{b_l}) = \tilde{\mathbf{Y}}_t^{b_l}. \end{cases} \tag{2}$$

This formulation shows that for samples from any task, the input features can be mapped to their corresponding outputs through *a single* optimal LoRA weight matrix $\tilde{\mathbf{A}}^{b_l}\tilde{\mathbf{B}}^{b_l}$, thereby ensuring accurate predictions.

However, the joint training objective in Eq. (1) critically relies on simultaneous access to datasets from all tasks, which is infeasible in exemplar-free continual learning to obtain $\tilde{\mathbf{A}}^{b_l}, \tilde{\mathbf{B}}^{b_l}$. In practical CL scenarios, only the dataset $\mathcal{D}_i$ is available for task $\mathcal{T}_i$. By training on $\mathcal{D}_i$, we can obtain *task-specific* LoRA weights $\mathbf{A}_i^{b_l}, \mathbf{B}_i^{b_l}$ and *task-specific* classifier weights $\mathbf{W}_i^h$ optimized for $\mathcal{T}_i$ alone. Within this CL paradigm, for the QKV-projection layer in the l-th ViT block, we denote the input and output features via task-specific training as $\mathbf{X}_i^{b_l}$ and $\mathbf{Y}_i^{b_l}$ for task $\mathcal{T}_i$ ($i \in \{1, \ldots, t\}$), as shown in Fig. 2. These features satisfy the following relationship with the trained task-specific LoRA weights $\mathbf{A}_i^{b_l}\mathbf{B}_i^{b_l}$ and the pre-trained QKV-projection weights $\mathbf{W}^{b_l}$:

$$\begin{cases} \mathbf{X}_1^{b_l}(\mathbf{W}^{b_l} + \mathbf{A}_1^{b_l}\mathbf{B}_1^{b_l}) = \mathbf{Y}_1^{b_l}, \\ \mathbf{X}_2^{b_l}(\mathbf{W}^{b_l} + \mathbf{A}_2^{b_l}\mathbf{B}_2^{b_l}) = \mathbf{Y}_2^{b_l}, \\ \qquad\qquad\qquad \vdots \\ \mathbf{X}_t^{b_l}(\mathbf{W}^{b_l} + \mathbf{A}_t^{b_l}\mathbf{B}_t^{b_l}) = \mathbf{Y}_t^{b_l}. \end{cases} \tag{3}$$

Note that the features $\mathbf{X}_i^{b_l}$ and $\mathbf{Y}_i^{b_l}$ are not identical to $\tilde{\mathbf{X}}_i^{b_l}$ and $\tilde{\mathbf{Y}}_i^{b_l}$, as they are obtained under task-specific and joint training paradigms, respectively. Compared to Eq. (2), the key difference is that the cross-task shared LoRA $\tilde{\mathbf{A}}^{b_l}\tilde{\mathbf{B}}^{b_l}$ is replaced by task-specific LoRAs $\mathbf{A}_i^{b_l}\mathbf{B}_i^{b_l}$. If the task identifier of an input sample is known during inference, the corresponding $\mathbf{A}_i^{b_l}\mathbf{B}_i^{b_l}$ can be directly applied (which we refer to as the *oracle upper bound*), yielding an exact mapping from $\mathbf{X}_i^{b_l}$ to $\mathbf{Y}_i^{b_l}$ and thus producing correct prediction. However, the task identifier is unknown and hard to predict reliably, leading to inconsistent input-output mappings and thereby degraded performance.

Inspired by Eq. (2), where a single set of parameters across all tasks enables a consistent input-output feature mapping, we propose a *unified optimization objective* for Eq. (3). In our approach, we reformulate CL as a feature mapping problem and replace the task-specific LoRAs with a single parameter matrix shared across tasks. This design aims to eliminate the need to infer task identities while ensuring that input features are mapped to their correct outputs. Specifically, we introduce a *unified adaptation matrix* $\mathbf{\Theta}_t^{b_l} \in \mathbb{R}^{D\times E}$ to replace the LoRAs in the l-th ViT block for the observed $1, 2, \cdots, t$ tasks. It is expected to simultaneously satisfy all the input-output feature relationships established by all task-specific LoRAs:

$$\begin{cases} \mathbf{X}_1^{b_l}(\mathbf{W}^{b_l} + \mathbf{\Theta}_t^{b_l}) = \mathbf{Y}_1^{b_l}, \\ \mathbf{X}_2^{b_l}(\mathbf{W}^{b_l} + \mathbf{\Theta}_t^{b_l}) = \mathbf{Y}_2^{b_l}, \\ \qquad\qquad\qquad \vdots \\ \mathbf{X}_t^{b_l}(\mathbf{W}^{b_l} + \mathbf{\Theta}_t^{b_l}) = \mathbf{Y}_t^{b_l}. \end{cases} \tag{4}$$

Since it is generally infeasible to find a matrix that exactly satisfies all equations, we relax the problem to minimizing the least-squares error across tasks (measured in the Frobenius norm) as our *unified optimization objective*:

$$\min_{\mathbf{\Theta}_t^{b_l}} J_t(\mathbf{\Theta}_t^{b_l}) \triangleq \sum_{i=1}^{t} \left\| \mathbf{X}_i^{b_l}\left(\mathbf{W}^{b_l} + \mathbf{\Theta}_t^{b_l}\right) - \mathbf{Y}_i^{b_l} \right\|_{\mathrm{F}}^2 + \lambda \left\| \mathbf{\Theta}_t^{b_l} \right\|_{\mathrm{F}}^2, \tag{5}$$

where $\lambda > 0$ is a small regularization coefficient (1×10^{-5} in our setting) introduced to improve numerical stability. As a convex optimization problem, this objective admits a closed-form solution via the normal equations [69]. The optimal solution $\mathbf{\Theta}_t^{b_l *}$ corresponds to the least-squares approximation of the mappings in Eq. (4). Therefore, solving this objective enables us to emulate the behavior of joint training.

In solving $\mathbf{\Theta}_t^{b_l *}$, we aim to develop an efficient approach that updates the solution progressively as new tasks arrive (*i.e.*, with t increasing). Each update should rely only on the data from the current task, without the need to store all samples or features from previous tasks. In the next subsection, we will demonstrate that such progressive update can be achieved by maintaining *cumulative statistics* of fixed dimensionality. We further establish two theoretical properties of our method in continual learning.

D. Progressive Solution via Cumulative Statistics

1) Derivation of Progressive Solution: To derive the closed-form solution, we first construct stacked input and output feature matrices as $\mathbf{X}_{1:t}^{b_l} = [\mathbf{X}_1^{b_l}; \mathbf{X}_2^{b_l}; \cdots; \mathbf{X}_t^{b_l}] \in \mathbb{R}^{(tN)\times D}$ and $\mathbf{Y}_{1:t}^{b_l} = [\mathbf{Y}_1^{b_l}; \mathbf{Y}_2^{b_l}; \cdots; \mathbf{Y}_t^{b_l}] \in \mathbb{R}^{(tN)\times E}$, where the semicolon "$[\cdot\,;\,\cdot]$" denotes vertical concatenation of matrices. Then the closed form of the solution $\mathbf{\Theta}_t^{b_l *}$ is given by [69]: $\mathbf{\Theta}_t^{b_l *} = \left(\mathbf{X}_{1:t}^{b_l \top}\mathbf{X}_{1:t}^{b_l} + \lambda\mathbf{I}\right)^{-1}\mathbf{X}_{1:t}^{b_l \top}\left(\mathbf{Y}_{1:t}^{b_l} - \mathbf{X}_{1:t}^{b_l}\mathbf{W}^{b_l}\right)$, where $\mathbf{I}$ denotes the identity matrix. While this formulation provides an exact closed-form solution, it requires storing all features from observed tasks, which is impractical for continual learning. To overcome this obstacle, we reformulate the solution in terms

of cumulative statistics that can be updated progressively as new tasks arrive, without revisiting historical samples.

Specifically, by expanding the stacked formulation into a summation over individual tasks, we obtain:

$$\mathbf{\Theta}_t^{b_l*} = \big(\sum_{i=1}^{t} \mathbf{X}_i^{b_l\top}\mathbf{X}_i^{b_l} + \lambda\mathbf{I}\big)^{-1}\big(\sum_{i=1}^{t} \mathbf{X}_i^{b_l\top}\big(\mathbf{Y}_i^{b_l} - \mathbf{X}_i^{b_l}\mathbf{W}^{b_l}\big)\big). \tag{6}$$

To facilitate continual derivation of the solution, we introduce two cumulative statistics that transform the summations into a progressive update form. The first cumulative term $\mathbf{F}_t^{b_l}$ is defined as the second-order statistics of the input features:

$$\mathbf{F}_t^{b_l} = \sum_{i=1}^{t} \mathbf{X}_i^{b_l\top}\mathbf{X}_i^{b_l} \in \mathbb{R}^{D\times D}. \tag{7}$$

This term corresponds to the accumulated uncentered covariance of the input features over the observed t tasks, capturing the overall structure of the input distribution. The second cumulative term $\mathbf{G}_t^{b_l}$ is the weighted residual statistic of the input-output pairs:

$$\mathbf{G}_t^{b_l} = \sum_{i=1}^{t} \mathbf{X}_i^{b_l\top}\left(\mathbf{Y}_i^{b_l} - \mathbf{X}_i^{b_l}\mathbf{W}^{b_l}\right) \in \mathbb{R}^{D\times E}. \tag{8}$$

$\mathbf{G}_t^{b_l}$ aggregates the input-weighted errors between the observed outputs and those predicted by the pre-trained weights. With these two definitions, the solution is given by:

$$\mathbf{\Theta}_t^{b_l*} = \big(\mathbf{F}_t^{b_l} + \lambda\mathbf{I}\big)^{-1}\mathbf{G}_t^{b_l}. \tag{9}$$

In this way, the knowledge of all past tasks is compactly summarized in the cumulative statistics $\mathbf{F}_t^{b_l}$ and $\mathbf{G}_t^{b_l}$.

After completing the training of the current task $\mathcal{T}_t$, the solution $\mathbf{\Theta}_t^{b_l*}$ can be computed without revisiting historical data. Using only the input features $\mathbf{X}_t^{b_l}$ and output features $\mathbf{Y}_t^{b_l}$ of the current task, we first update the cumulative statistics progressively as follows:

$$\begin{cases} \mathbf{F}_t^{b_l} = \mathbf{F}_{t-1}^{b_l} + \mathbf{X}_t^{b_l\top}\mathbf{X}_t^{b_l}, & \text{(10a)} \\ \mathbf{G}_t^{b_l} = \mathbf{G}_{t-1}^{b_l} + \mathbf{X}_t^{b_l\top}\left(\mathbf{Y}_t^{b_l} - \mathbf{X}_t^{b_l}\mathbf{W}^{b_l}\right). & \text{(10b)} \end{cases}$$

The update terms introduced by the new task are highlighted in blue in the above equations. They depend solely on the input and output features from the current task, without involving the outputs of previous tasks $\mathbf{Y}_j^{b_l}(j<t)$. This indicates that even if the training of LoRA on a new task causes output shifts in the old data, the resulting solution can still align their outputs with the corresponding original target features rather than the drifted ones. Subsequently, the new solution $\mathbf{\Theta}_t^{b_l*}$ can be computed by multiplying the inverse of $\mathbf{F}_t^{b_l}$ with $\mathbf{G}_t^{b_l}$ (as in Eq. (9)). As a result, the unified adaptation matrix is obtained progressively in an online manner, eliminating the need to store or revisit raw data from previous tasks.

For notational simplicity, the above derivation is presented at the single-sample level. In practice, when each task contains multiple samples, the statistics are constructed by aggregating over all samples within each task, *i.e.*, $\mathbf{F}_t = \sum_{i=1}^{t}\sum_{n=1}^{|\mathcal{T}_i|}\mathbf{X}_i^{(n)\top}\mathbf{X}_i^{(n)}$ and $\mathbf{G}_t = \sum_{i=1}^{t}\sum_{n=1}^{|\mathcal{T}_i|}\mathbf{X}_i^{(n)\top}(\mathbf{Y}_i^{(n)} - \mathbf{X}_i^{(n)}\mathbf{W})$, where n indexes samples in task $\mathcal{T}_i$ and the layer superscript b_l is omitted for brevity. In our implementation, these statistics are accumulated in a batch-wise manner, which is mathematically equivalent to summing over all samples.

Our proposed strategy offers two key advantages: (a) *Plasticity-stability trade-off*: the unified adaptation matrix $\mathbf{\Theta}_t^{b_l*}$ minimizes the mean squared fitting error over all tasks, thereby achieving an effective and automatic balance between plasticity and stability under an appropriate criterion. (b) *Storage efficiency*: the sizes of $\mathbf{F}_t^{b_l} \in \mathbb{R}^{D\times D}$ and $\mathbf{G}_t^{b_l} \in \mathbb{R}^{D\times E}$ depend solely on the feature dimensions and remain constant regardless of the number of tasks, classes or samples, making the method suitable for resource-constrained scenarios.

At the inference stage, the LoRA module of the l-th layer ($\mathbf{A}_t^{b_l}\mathbf{B}_t^{b_l}$) is replaced by the computed unified adaptation matrix $\mathbf{\Theta}_t^{b_l*}$, and thus the QKV-projection layer is given by ($\mathbf{W}^{b_l}$+$\mathbf{\Theta}_t^{b_l*}$). In practice, our method cannot perfectly achieve the effect of joint training due to inevitable approximation errors. Specifically, the objective in Eq. (5) cannot be minimized to zero, and small deviations accumulate across subsequent layers. As a result, the mapping from $\mathbf{X}_i^{b_l}$ to $\mathbf{Y}_i^{b_l}$ in Eq. (4) is satisfied approximately rather than exactly. Nevertheless, these errors are minor and do not substantially affect performance, allowing our approach to achieve state-of-the-art results.

2) Theoretical Properties: Achieving an appropriate trade-off in the stability-plasticity dilemma is a central challenge in continual learning. To provide theoretical guarantees for our method, we formally establish two key properties: (1) the inherent balance between stability and plasticity, and (2) the monotonic decrease of the unified optimization objective under solution updates. For brevity, the superscript b_l used in earlier definitions is omitted in the following derivations. The detailed proofs are provided in the Appendix.

Property 1 (Stability-Plasticity Balance)**.** *Let $\mathbf{\Theta}_{t-1}^*$ and $\mathbf{\Theta}_t^*$ denote the unified adaptation matrices for tasks $\mathcal{T}_{t-1}$ and $\mathcal{T}_t$, respectively, and define the parameter update as $\Delta\mathbf{\Theta}_t = \mathbf{\Theta}_t^* - \mathbf{\Theta}_{t-1}^*$. For the old-task objective $J_{\text{old}}(\mathbf{\Theta}) = J_{t-1}(\mathbf{\Theta})$, and the single new-task objective $J_{\text{new}}(\mathbf{\Theta}) = J_t(\mathbf{\Theta}) - J_{\text{old}}(\mathbf{\Theta})$, the following relations hold:*

$$\begin{aligned} \Delta J_{\text{old}} &= J_{\text{old}}(\mathbf{\Theta}_t^*) - J_{\text{old}}(\mathbf{\Theta}_{t-1}^*) \\ &= \|\,(\mathbf{F}_{t-1} + \lambda\mathbf{I})^{\frac{1}{2}}\,\Delta\mathbf{\Theta}_t\,\|_{\text{F}}^2 \;\geq 0, & \text{(11)} \\ \Delta J_{\text{new}} &= J_{\text{new}}(\mathbf{\Theta}_t^*) - J_{\text{new}}(\mathbf{\Theta}_{t-1}^*) \\ &= -2\|(\mathbf{F}_{t-1}+\lambda\mathbf{I})^{\frac{1}{2}}\Delta\mathbf{\Theta}_t\|_{\text{F}}^2 - \|\mathbf{X}_t\Delta\mathbf{\Theta}_t\|_{\text{F}}^2 \leq 0, & \text{(12)} \\ |\Delta J_{\text{old}}| &\leq \tfrac{1}{2}\,|\Delta J_{\text{new}}|. & \text{(13)} \end{aligned}$$

Eq. (11) indicates that the loss over previous tasks inevitably increases after the update of unified adaptation matrix, which reflects the phenomenon of forgetting. Eq. (12) demonstrates that the loss on the newly arrived task will decrease, highlighting the model's plasticity. Finally, Eq. (13) quantifies the stability-plasticity trade-off: the increase in the old-task loss is rigorously upper bounded by one half of the absolute decrease in the new-task loss. It ensures that the adaptation to new tasks does not incur excessive forgetting of previously learned tasks.

Property 2 (Monotonic Objective Decrease under Solution Updates)**.** *Let $J_t(\mathbf{\Theta})$ be the optimization objective defined in Eq.* (5)*. Then the update from $\mathbf{\Theta}^*_{t-1}$ to $\mathbf{\Theta}^*_t$ satisfies:*

$$J_t(\mathbf{\Theta}^*_t) - J_t(\mathbf{\Theta}^*_{t-1}) = -\|(\mathbf{F}_t + \lambda\mathbf{I})^{1/2}\,\Delta\mathbf{\Theta}_t\,\|^2_{\mathrm{F}} \leq 0. \quad (14)$$

Eq. (14) provides two insights into our progressive optimization. First, the update from $\mathbf{\Theta}^*_{t-1}$ to $\mathbf{\Theta}^*_t$ guarantees a non-increasing loss ($J_t(\mathbf{\Theta}^*_t) \leq J_t(\mathbf{\Theta}^*_{t-1})$), implying that each solution update moves the unified adaptation matrix closer to the joint optimum over tasks $\mathcal{T}_1, \mathcal{T}_2, \ldots, \mathcal{T}_t$. Second, the amount of decrease is quantified as $\|(\mathbf{F}_t + \lambda\mathbf{I})^{1/2}\Delta\mathbf{\Theta}_t\|^2_{\mathrm{F}}$, which corresponds to the parameter update $\Delta\mathbf{\Theta}_t$ measured in the Mahalanobis norm defined by the cumulative uncentered covariance matrix $\mathbf{F}_t$.

In addition, Eq. (5) can also be interpreted from a probabilistic perspective as a maximum a posteriori (MAP) estimator under Gaussian likelihood and Gaussian prior assumptions [70], which is provided in Section III in the Appendix.

E. Progressive Solution for Classifier

In contrast to the LoRA modules in backbone, the classifier presents unique challenges in continual learning: each task trains its classifier independently, whereas inference requires combining all task-specific classifiers. This leads to 1) an inconsistency of the input-output mappings between training and inference stages, 2) cross-task confusion due to the absence of training between the features and classifiers originated from different tasks, and 3) expanding output dimensions preventing direct reuse of the backbone's progressive update rule in Eq. (10) for updating solutions. In this subsection, we provide a complete analysis of the unified optimization for the classifier. By incorporating the global classifier with extended output space, the approximation for undefined cross-task logits and additional cumulative statistics, we ultimately derive a solution that enables progressive updates of the classifier's unified adaptation matrix.

For the classifier of the i-th task, the trained weight matrix is denoted as $\mathbf{W}^h_i \in \mathbb{R}^{D\times C_i}$, and the input feature from this task is $\mathbf{X}^h_i \in \mathbb{R}^{1\times D}$. After training the classifier in task $\mathcal{T}_i$, the input features and output logits satisfy:

$$\mathbf{X}^h_i\mathbf{W}^h_i = \hat{\mathbf{H}}_i, \quad (15)$$

where $\hat{\mathbf{H}}_i \in \mathbb{R}^{1\times C_i}$ is the predicted within-task logits and is expected to match the ground-truth logits $\mathbf{H}_i$ of task $\mathcal{T}_i$.

To establish a consistent feature mapping across all tasks, we extend the output space to cover all available classes. The global classifier weight matrix is defined by concatenating all task-specific weights:

$$\mathbf{W}^{h_{(t)}} = [\mathbf{W}^h_1, \mathbf{W}^h_2, \cdots, \mathbf{W}^h_t] \in \mathbb{R}^{D\times C_{(t)}}, \quad (16)$$

where $C_{(t)} = \sum_{i=1}^{t} C_i$ denotes the total number of available classes up to task $\mathcal{T}_t$, and "$[\cdot,\cdot]$" indicates the horizontal concatenation of matrices. Accordingly, the predicted logits output by the global classifier is:

$$\hat{\mathbf{Y}}^{h_{(t)}}_i = \mathbf{X}^h_i\mathbf{W}^{h_{(t)}} = [\hat{\mathbf{Z}}_{i,1}, \hat{\mathbf{Z}}_{i,2}, \cdots, \hat{\mathbf{Z}}_{i,t}] \in \mathbb{R}^{1\times C_{(t)}}, \quad (17)$$

where $\hat{\mathbf{Z}}_{i,j}$ denotes the predicted logits of the j-th task's classifier $\mathbf{W}^h_j$ for the input features $\mathbf{X}^h_i$ from task $\mathcal{T}_i$. In particular, both the features and the classifier from the i-th task satisfy $\hat{\mathbf{Z}}_{i,i} = \hat{\mathbf{H}}_i$.

However, the predictions of the classifier still suffer from two issues: 1) *within-task error*, where the classifier weights $\mathbf{W}^h_i$ cannot fully fit the classes within task $\mathcal{T}_i$, and 2) *cross-task confusion*, where $\mathbf{W}^h_i$ fails to provide reliable predictions for features from other tasks $\mathbf{X}^h_j (j \neq i)$, since it is trained solely on the features of task $\mathcal{T}_i$. Therefore, it is necessary to calibrate the output logits of the classifier so that they align with the expected ground-truth logits. Similar to Eq. (5), we aim to solve for a unified adaptation matrix $\mathbf{\Theta}^h_t \in \mathbb{R}^{D\times C_{(t)}}$ that minimizes the error between the output logits and the ground-truth logits $\mathbf{Y}^{h_{(t)}}_i$:

$$\min_{\mathbf{\Theta}^h_t} \sum_{i=1}^{t} \left\|\mathbf{X}^h_i\left(\mathbf{W}^{h_{(t)}} + \mathbf{\Theta}^h_t\right) - \mathbf{Y}^{h_{(t)}}_i\right\|^2_{\mathrm{F}} + \lambda\left\|\mathbf{\Theta}^h_t\right\|^2_{\mathrm{F}}. \quad (18)$$

The optimal solution to Eq. (18) is derived as:

$$\mathbf{\Theta}^{h*}_t = \left(\mathbf{F}^h_t + \lambda\mathbf{I}\right)^{-1}\mathbf{G}^h_t, \quad (19)$$

where:

$$\begin{cases} \mathbf{F}^h_t = \sum_{i=1}^{t} \mathbf{X}^{h\top}_i\mathbf{X}^h_i \in \mathbb{R}^{D\times D}, & (20a) \\ \mathbf{G}^h_t = \sum_{i=1}^{t} \mathbf{X}^{h\top}_i(\mathbf{Y}^{h_{(t)}}_i - \mathbf{X}^h_i\mathbf{W}^{h_{(t)}}) \in \mathbb{R}^{D\times C_{(t)}}. & (20b) \end{cases}$$

$\mathbf{F}^h_t$ in Eq. (20a) can be progressively updated in the same form as Eq. (10a), *i.e.*, $\mathbf{F}^h_t = \mathbf{F}^h_{t-1} + \mathbf{X}^{h\top}_t\mathbf{X}^h_t$. However, Eq. (20b) cannot follow the same update rule of Eq. (10b), since the output dimension of $\mathbf{G}^h_t$ increases with the number of tasks ($C_{(t-1)} \to C_{(t)}$). This dimensionality growth causes a mismatch between the previous cumulative statistics $\mathbf{G}^h_{t-1}$ and the current term $\mathbf{X}^{h\top}_t(\mathbf{Y}^{h_{(t)}}_t - \mathbf{X}^h_t\mathbf{W}^{h_{(t)}})$, which prevents direct accumulation. Therefore, deriving an effective progressive update for $\mathbf{\Theta}^{h*}_t$ remains a challenge for the classifier.

To address this issue, we further decompose $\mathbf{G}^h_t$ into two intermediate cumulative statistics $\mathbf{P}^{h_{(t)}}_t$ and $\mathbf{Q}^{h_{(t)}}_t$:

$$\begin{cases} \mathbf{P}^{h_{(t)}}_t = \sum_{i=1}^{t} \mathbf{X}^{h\top}_i\mathbf{Y}^{h_{(t)}}_i, & (21a) \\ \mathbf{Q}^{h_{(t)}}_t = \sum_{i=1}^{t} \mathbf{X}^{h\top}_i\mathbf{X}^h_i\mathbf{W}^{h_{(t)}}, & (21b) \end{cases}$$

such that:

$$\mathbf{G}^h_t = \mathbf{P}^{h_{(t)}}_t - \mathbf{Q}^{h_{(t)}}_t. \quad (22)$$

For brevity, we introduce the notation $\bar{\mathbf{X}}^h_i = \mathbf{X}^{h\top}_i\mathbf{X}^h_i \in \mathbb{R}^{D\times D}$. After derivation (provided in the Appendix), we obtain the progressive update rule for $\mathbf{Q}^{h_{(t)}}_t$ as follows, where the terms highlighted in blue indicate the additional components required when updating from $\mathbf{Q}^{h_{(t-1)}}_{t-1}$ to $\mathbf{Q}^{h_{(t)}}_t$ ($t > 1$):

$$\mathbf{Q}^{h_{(t)}}_t = \big[\mathbf{F}^h_{t-1}\mathbf{W}^h_1 + \bar{\mathbf{X}}^h_t\mathbf{W}^h_1, \quad \cdots, \quad \mathbf{F}^h_{t-1}\mathbf{W}^h_{t-1} + \bar{\mathbf{X}}^h_t\mathbf{W}^h_{t-1}, \quad \mathbf{F}^h_{t-1}\mathbf{W}^h_t + \bar{\mathbf{X}}^h_t\mathbf{W}^h_t\big]. \quad (23)$$

This formula implies that $\mathbf{Q}^{h_{(t)}}_t$ can be updated incrementally from $\mathbf{Q}^{h_{(t-1)}}_{t-1}$ by: (1) adding $\bar{\mathbf{X}}^h_t$ multiplied by the corresponding $\mathbf{W}^h_i$ to each block of $\mathbf{Q}^{h_{(t-1)}}_{t-1}$ (equivalently,

$\mathbf{Q}_{t-1}^{h_{(t-1)}}+\bar{\mathbf{X}}_t^h\mathbf{W}^{h_{(t-1)}}$), and (2) appending a new block formed by $\mathbf{F}_{t-1}^h\mathbf{W}_t^h+\bar{\mathbf{X}}_t^h\mathbf{W}_t^h=\mathbf{F}_t^h\mathbf{W}_t^h$.

Similarly, the update of $\mathbf{P}_t^{h_{(t)}}$ is given by:

$$\mathbf{P}_t^{h_{(t)}}=\big[\sum_{i=1}^{t-1}\mathbf{X}_i^{h\top}\mathbf{Z}_{i,1}+\mathbf{X}_t^{h\top}\mathbf{Z}_{t,1},\quad\cdots,\quad\sum_{i=1}^{t-1}\mathbf{X}_i^{h\top}\mathbf{Z}_{i,t-1}+\mathbf{X}_t^{h\top}\mathbf{Z}_{t,t-1},\ \sum_{i=1}^{t-1}\mathbf{X}_i^{h\top}\mathbf{Z}_{i,t}+\mathbf{X}_t^{h\top}\mathbf{Z}_{t,t}\big]. \tag{24}$$

However, the ground-truth logits $\mathbf{Z}_{i,j}$ generated by features of task $\mathcal{T}_i$ on the classifier of task $\mathcal{T}_j$ ($i\neq j$) are inaccessible, which hinders the implementation of Eq. (24). To address this issue, we propose to approximate $\mathbf{Z}_{i,j}$ ($j\neq i$) by the scaling of the predicted logits: $\alpha\hat{\mathbf{Z}}_{i,j}$. The factor $0<\alpha<1$ enforces that the output logits from non-target classifiers remain smaller than those from the correct i-th classifier, thereby calibrating the predicted scores and alleviating cross-task confusion. For the case where features and classifiers belong to the same task ($\mathbf{Z}_{i,i}$), in order to reduce within-task error, we do not adopt the predicted logits $\hat{\mathbf{H}}_i$ as the ground-truth logits. Instead, we define the ground-truth logits $\mathbf{H}_i$ as a scaled one-hot vector $\beta\mathbf{e}_c\in\mathbb{R}^{1\times C_i}$, where c denotes the true class index and $\mathbf{e}_c$ is the standard basis vector with a 1 at index c and 0 elsewhere. $\beta\gg1$ serves as a magnification factor to enforce a larger margin for more reliable within-task classification. The resulting target output logits are summarized as:

$$\mathbf{Z}_{i,j}=\begin{cases}\alpha\hat{\mathbf{Z}}_{i,j}=\alpha\mathbf{X}_i^h\mathbf{W}_j^h\in\mathbb{R}^{1\times C_j},\ \text{if } j\neq i, & \text{(25a)}\\ \mathbf{H}_i=\beta\mathbf{e}_c\in\mathbb{R}^{1\times C_i},\ \ \text{if } j=i. & \text{(25b)}\end{cases}$$

This scaled-logit approximation, rather than simply setting $\mathbf{Z}_{i,j}=0$ for $j\neq i$ [30], [63]–[65], accounts for the bias of different classifier outputs and reduces the fitting error of the optimization objective. It also facilitates the simplification required for progressive updates, which enables us to obtain the following expression for $\mathbf{P}_t^{h_{(t)}}$:

$$\mathbf{P}_t^{h_{(t)}}=\big[\alpha\sum_{i\neq1}^{t-1}\bar{\mathbf{X}}_i^h\mathbf{W}_1^h+\mathbf{X}_1^{h\top}\mathbf{H}_1+\alpha\bar{\mathbf{X}}_t^h\mathbf{W}_1^h,\quad\cdots,\quad\alpha\sum_{i=1}^{t-2}\bar{\mathbf{X}}_i^h\mathbf{W}_{t-1}^h+\mathbf{X}_{t-1}^{h\top}\mathbf{H}_{t-1}+\alpha\bar{\mathbf{X}}_t^h\mathbf{W}_{t-1}^h,\quad\alpha\mathbf{F}_{t-1}^h\mathbf{W}_t^h+\mathbf{X}_t^{h\top}\mathbf{H}_t\big]. \tag{26}$$

This formulation shows that the first $t-1$ blocks of $\mathbf{P}_t^{h_{(t)}}$ follow the similar update rule as $\mathbf{Q}_t^{h_{(t)}}$ with an additional coefficient α. The newly appended block is obtained by multiplying the previous cumulative statistic $\mathbf{F}_{t-1}^h$ with the current classifier weight $\mathbf{W}_t^h$ and adding $\mathbf{X}_t^{h\top}\mathbf{H}_t$. Together with $\mathbf{Q}_t^{h_{(t)}}$, this enables the progressive update of $\mathbf{G}_t^h$ in Eq. (20b), thereby completing the solution of the unified adaptation matrix in Eq. (19) for the global classifier. At the inference stage, we adopt $(\mathbf{W}^{h_{(t)}}+\mathbf{\Theta}_t^{h*})$ as the weights of the global classifier, enabling calibrated prediction across all learned classes. Moreover, the procedure remains memory-efficient: only $\mathbf{F}_t^h\in\mathbb{R}^{D\times D}$ and $\mathbf{P}_t^{h_{(t)}}\in\mathbb{R}^{D\times C_{(t)}}$ need to be stored, while $\mathbf{Q}_t^{h_{(t)}}$ in Eq. (23) can be computed on the fly from $\mathbf{F}_{t-1}^h$ and the classifier weights, which thereby eliminates the need for storage.

Algorithm 1 The proposed cumulative unified optimization framework for continual low-rank adaptation

Require: Task sequence $\mathcal{T}_1,\cdots,\mathcal{T}_T$ with datasets $\mathcal{D}_1,\cdots,\mathcal{D}_T$, where $\mathcal{D}_t=\{(x_t^{(n)},y_t^{(n)})\}_{n=1}^{|\mathcal{T}_t|}$; ViT model $f(\cdot)$ with LoRA parameters $\mathbf{A}_t^{b_l},\mathbf{B}_t^{b_l}$ for layer $l\in\{1,\cdots,L\}$ and task-specific classifier $\mathbf{W}_t^h$; cumulative statistics $\mathbf{F}_t^{b_l},\mathbf{G}_t^{b_l},\mathbf{F}_t^h$, and $\mathbf{P}_t^{h_{(t)}}$
Ensure: Unified backbone adaptations $\{\mathbf{\Theta}_t^{b_l*}\}_{l=1}^L$ and classifier adaptation $\mathbf{\Theta}_t^{h*}$

Initialization: $\mathbf{F}_0^{b_l},\mathbf{G}_0^{b_l},\mathbf{F}_0^h,\mathbf{P}_0^{h_{(0)}}\leftarrow\mathbf{0}$
for task $t=1$ to T **do**
  // Training stage
  if $t>1$ **then**
    $\mathbf{A}_t^{b_l},\mathbf{B}_t^{b_l}\leftarrow$ Eq. (27) for each block l
  end if
  repeat
    Sample $(\boldsymbol{x}_t,\boldsymbol{y}_t)\sim\mathcal{D}_t$
    $\hat{\boldsymbol{y}}_t\leftarrow f(\boldsymbol{x}_t;\{\mathbf{A}_t^{b_l},\mathbf{B}_t^{b_l}\}_{l=1}^L,\mathbf{W}_t^h)$
    $\ell\leftarrow\text{CrossEntropy}(\hat{\boldsymbol{y}}_t,\boldsymbol{y}_t)$
    Update $\{\mathbf{A}_t^{b_l},\mathbf{B}_t^{b_l}\}_{l=1}^L$ and $\mathbf{W}_t^h$ by backpropagation
  until convergence on $\mathcal{D}_t$
  for layer $l=1$ to L **do**
    Extract $\mathbf{X}_t^{b_l},\mathbf{Y}_t^{b_l},\mathbf{W}^{b_l}$ from the QKV-projection layer
    $\mathbf{F}_t^{b_l},\mathbf{G}_t^{b_l}\leftarrow$ Eq. (10)
    $\mathbf{\Theta}_t^{b_l*}\leftarrow$ Eq. (9)
  end for
  Extract $\mathbf{X}_t^h$; $\mathbf{Z}_{i,j}\leftarrow$ Eq. (25)
  $\mathbf{F}_t^h\leftarrow$ Eq. (20a); $\mathbf{P}_t^{h_{(t)}}\leftarrow$ Eq. (26)
  $\mathbf{Q}_t^{h_{(t)}}\leftarrow$ Eq. (23); $\mathbf{G}_t^h\leftarrow$ Eq. (22)
  $\mathbf{\Theta}_t^{h*}\leftarrow$ Eq. (19)
  // Inference stage
  `LoRA module` $\mathbf{A}_t^{b_l}\mathbf{B}_t^{b_l}\leftarrow\mathbf{\Theta}_t^{b_l*}$
  `Global classifier` $\leftarrow(\mathbf{W}^{h_{(t)}}+\mathbf{\Theta}_t^{h*})$
  Predict with the updated model
end for

Existing analytic analyses of classifiers [30], [63]–[65] can be viewed as special cases of our framework under the restrictive assumption $\alpha=0$ for $j\neq i$ and $\mathbf{Z}_{i,j}=\hat{\mathbf{H}}_i=\mathbf{X}_i^h\mathbf{W}_i^h$ for $j=i$ in Eq. (25). Setting $\alpha=0$ enforces all target cross-task logits to be zero, ignoring the varying difficulty of fitting input-output relationships across different tasks. As a result, the solution cannot fully capture the true mappings, leading to suboptimal cross-task correction. $\mathbf{Z}_{i,i}=\mathbf{X}_i^h\mathbf{W}_i^h$ means that no correction is applied to the within-task logits, which results in severe within-task errors. In contrast, our formulation provides a more generalized analysis that incorporates cross-task logits while also accounting for within-task correction, thereby mitigating both inter-task and intra-task errors and achieving superior empirical performance, as demonstrated in our experiments.

F. Forward Transfer Initialization

In continual learning, the LoRA parameters for new tasks are typically initialized randomly or inherited directly from the parameters of the previous tasks. However, this approach fails to effectively leverage the knowledge acquired from historical tasks, especially in scenarios with scarce samples, leading to unstable training or slow convergence. To address

this limitation, it is necessary to provide the new task with a high-quality initialization that absorbs the common knowledge shared across historical tasks, thereby enabling forward knowledge transfer across tasks.

An intuitive approach is to reuse the computed unified adaptation parameters $\mathbf{\Theta}_t^{b_l*}$, which aggregates the information of all observed tasks. However, while $\mathbf{\Theta}_t^{b_l*}$ is a full-rank matrix of size $D\times E$, the LoRA parameters to be initialized for the new task $\mathcal{T}_{t+1}$ (denoted as $\dot{\mathbf{A}}_{t+1}^{b_l}, \dot{\mathbf{B}}_{t+1}^{b_l}$) adopt a low-rank factorization form $(D\times r, r\times E)$, which introduces a dimensional mismatch. Forcing an SVD-based low-rank approximation of $\mathbf{\Theta}_t^{b_l*}$ (by truncating lower singular values) induces uncontrollable approximation errors and numerical instability, which degrades the training process. Instead, we aim to obtain a reliable estimate of the initialization parameters for the new task from the distribution of all previously trained LoRA parameters. In practice, we realize this estimate by taking the arithmetic mean of all previously trained LoRA parameters. Specifically, after completing the training of task $\mathcal{T}_i$ $(i\leq t)$, we can obtain a set of LoRA parameters $\{\mathbf{A}_i^{b_l}, \mathbf{B}_i^{b_l}\}_{i=1}^t$ for each block b_l. For the subsequent task $\mathcal{T}_{t+1}$, we initialize the new LoRA parameters as the average of all previously trained LoRAs:

$$\dot{\mathbf{A}}_{t+1}^{b_l} = \frac{1}{t}\sum_{i=1}^{t}\mathbf{A}_i^{b_l}, \quad \dot{\mathbf{B}}_{t+1}^{b_l} = \frac{1}{t}\sum_{i=1}^{t}\mathbf{B}_i^{b_l}. \tag{27}$$

The initialized $\dot{\mathbf{A}}_{t+1}^{b_l}$ and $\dot{\mathbf{B}}_{t+1}^{b_l}$ are further optimized through task-specific training to obtain the final parameters $\mathbf{A}_{t+1}^{b_l}$ and $\mathbf{B}_{t+1}^{b_l}$ for $\mathcal{T}_{t+1}$. This strategy offers notable improvements in long-sequence scenarios, where limited samples often lead to instability and slow convergence. Moreover, it is storage-efficient with a constant memory cost, since the arithmetic mean can be efficiently calculated through an online averaging scheme without retaining all historical parameters. Algorithm 1 presents the detailed procedure for our full method.

V. Experiments

A. Experimental Setups

1) Benchmarks: We evaluate our approach under the class-incremental learning (CIL) protocol, where the classes for each task are mutually exclusive and task identifiers are not provided at test time. Experiments are conducted on four widely used benchmarks: (a) *ImageNet-R* [71] consists of 200 classes with diverse styles such as paintings, embroidery, and cartoons. We randomly split these classes into 10 or 20 tasks (referred to as 10/20-split ImageNet-R) with an equal number of classes per task to form two CIL benchmarks. (b) *CIFAR-100* [72] is a classic natural image classification dataset containing 100 classes, which we divide into 10 tasks for the 10-split benchmark. (c) *DomainNet* [73] is a large multi-domain classification dataset comprising six distinct visual domains (*e.g.*, paintings, sketches, cliparts) and originally 345 categories. Following the CIL setting proposed in [41], [44], [49], we select the 200 classes with the largest number of images to construct a challenging cross-domain CIL benchmark, where each task contains 20 classes spanning the 6 domains. (d) *ImageNet-A* [74] contains 200 classes of images that are particularly challenging for standard models, and we split these classes into 20 tasks to evaluate the robustness of methods. In addition to these conventional CL settings, we further split ImageNet-R and DomainNet into 50 and 100 tasks respectively, and CIFAR-100 into 50 tasks, to assess performance under long-sequence CL scenarios.

2) Implementation Details: We employ the ViT-Base/16 [2] architecture pre-trained on the ImageNet-21K dataset [75] as our backbone network. LoRA modules with a rank of $r = 8$ are inserted into the QKV-projection layers of all 12 Transformer blocks. During training, only these LoRA modules and the task-specific classifiers are optimized, while all remaining backbone parameters are kept frozen.

For all experiments, input images are resized to 224×224 pixels and augmented with AutoAugment [76] during training. We use the Adam optimizer [77] with a weight decay of 5×10^{-5}. The learning rate is set to 0.0001 for the LoRA modules and 0.01 for the classifiers. Each task is trained for 100 epochs with a batch size of 256, and the learning rate is decayed by a factor of 0.1 at the 50-th and 80-th epochs. The model is trained solely using cross-entropy loss with a softmax temperature of 30. Our method involves three hyper-parameters: the regularization coefficient λ in Eq. (5) and (18) is set to 1×10^{-5}, and the scaling factors α and β in Eq. (25) are set to 0.3 and 200, respectively. All results are averaged over three independent runs with different random seeds.

Our approach is implemented in PyTorch [78] using the timm [79] library. All experiments are conducted on a server equipped with 128 GB RAM and 4 NVIDIA RTX 4090 GPUs.

3) Evaluation Metrics: We evaluate the performance of methods using two widely used metrics following prior CL works [18]–[20], *i.e.*, *final average accuracy* and *final average forgetting*. They are defined as: $Accuracy = \frac{1}{T}\sum_{i=1}^{T} a_{T,i}$, $Forgetting = \frac{1}{T-1}\sum_{i=1}^{T-1}\max_{j\in\{1,\dots,T-1\}}\big(a_{j,i}-a_{T,i}\big)$. Here T denotes the total number of tasks. The value $a_{T,i}$ is the accuracy of the final model on the test set of task $\mathcal{T}_i$ after training on all T tasks. The value $a_{j,i}$ is the accuracy on the i-th test set immediately after the model finishes training on task $\mathcal{T}_j$. Following common practice in parameter-efficient CL, the final average accuracy is used as the primary metric for performance comparison. Although some studies additionally report the mean accuracy across all tasks, for consistency we adopt the final average accuracy as the only accuracy metric throughout this paper.

A higher final average accuracy reflects stronger overall classification capability, whereas lower forgetting indicates greater stability in retaining learned knowledge. However, low forgetting alone does not guarantee good performance, since accuracy also depends on the model's ability to acquire new knowledge. Therefore, we emphasize the final average accuracy as the primary metric, as it provides the most direct measure of practical classification performance.

B. Comparison with Existing Methods

We compare our approach with a wide range of parameter-efficient CL methods under both conventional and long-sequence settings. Firstly, we establish a sequential fine-tuning baseline (Seq-FT), which directly employs the LoRA

TABLE I
COMPARISON WITH EXISTING METHODS. VALUES AFTER ± INDICATE STANDARD DEVIATIONS. "SEQ-FT" DENOTES SEQUENTIAL FINE-TUNING OF LORA MODULES. THE BEST ACCURACY AMONG ALL METHODS IS SHOWN IN **BOLD**, AND THE BEST AMONG EXISTING METHODS IS UNDERLINED.

Method	Venue	10S-ImageNet-R		20S-ImageNet-R		10S-CIFAR-100		10S-DomainNet		20S-ImageNet-A	
		Acc.↑	Forgetting↓	Acc.↑	Forgetting↓	Acc.↑	Forgetting↓	Acc.↑	Forgetting↓	Acc.↑	Forgetting↓
L2P [18]	CVPR'22	$61.57_{\pm 0.66}$	$9.73_{\pm 0.47}$	$59.38_{\pm 0.50}$	$5.89_{\pm 0.36}$	$83.83_{\pm 0.04}$	$7.63_{\pm 0.30}$	$81.17_{\pm 0.83}^{\dagger}$	$8.98_{\pm 1.25}$	$42.40^{\ddagger}$	-
DualPrompt [19]	ECCV'22	$68.13_{\pm 0.49}$	$4.68_{\pm 0.20}$	$63.21_{\pm 0.49}$	$5.28_{\pm 0.45}$	$86.51_{\pm 0.33}$	$5.16_{\pm 0.09}$	$81.70_{\pm 0.78}^{\dagger}$	$8.04_{\pm 0.31}$	$43.38^{\ddagger}$	-
CODA-Prompt [20]	CVPR'23	$75.45_{\pm 0.56}$	$1.64_{\pm 0.10}$	$72.37_{\pm 1.19}$	$0.96_{\pm 0.15}$	$86.25_{\pm 0.74}$	$1.67_{\pm 0.26}$	$80.04_{\pm 0.79}^{\dagger}$	$10.16_{\pm 0.35}$	$46.15^{\ddagger}$	-
APG [43]	ICCV'23	73.27	8.59	71.22	7.39	89.35	6.01	-	-	-	-
LAE [61]	ICCV'23	$72.66_{\pm 0.63}$	-	$69.67_{\pm 0.86}$	-	$85.59_{\pm 0.46}$	-	-	-	-	-
ESN [41]	AAAI'23	$71.07_{\pm 0.29}$	$4.99_{\pm 0.49}$	$64.77_{\pm 0.71}$	$6.65_{\pm 1.24}$	$86.34_{\pm 0.52}$	$4.76_{\pm 0.14}$	$79.22_{\pm 2.04}^{\dagger}$	$10.62_{\pm 2.12}$	-	-
OS-Prompt++ [45]	ECCV'24	$75.67_{\pm 0.40}$	$1.27_{\pm 0.10}$	$73.77_{\pm 0.19}$	$0.79_{\pm 0.07}$	$86.68_{\pm 0.67}$	$1.18_{\pm 0.21}$	-	-	-	-
EvoPrompt [21]	AAAI'24	$76.83_{\pm 0.08}$	$2.78_{\pm 0.06}$	$74.41_{\pm 0.23}$	$2.56_{\pm 0.22}$	$87.97_{\pm 0.30}$	$2.60_{\pm 0.42}$	$79.50_{\pm 0.29}$	$3.81_{\pm 0.36}$	-	-
PGP [57]	ICLR'24	$69.34_{\pm 0.05}$	$4.53_{\pm 0.04}$	-	-	$86.92_{\pm 0.05}$	$5.35_{\pm 0.19}$	$80.41_{\pm 0.25}$	$8.39_{\pm 0.18}$	-	-
OVOR-Deep [42]	ICLR'24	$76.11_{\pm 0.21}$	$7.16_{\pm 0.34}$	$73.85_{\pm 0.29}$	$6.80_{\pm 0.65}$	$85.99_{\pm 0.89}$	$6.42_{\pm 2.03}$	$79.61_{\pm 0.86}$	$4.77_{\pm 0.94}$	-	-
ConvPrompt [48]	CVPR'24	$77.86_{\pm 0.25}$	$4.33_{\pm 0.24}$	$75.10_{\pm 0.39}$	$4.10_{\pm 0.29}$	$88.87_{\pm 0.33}$	$4.75_{\pm 0.15}$	$79.47_{\pm 0.35}$	$6.49_{\pm 0.43}$	-	-
InfLoRA [31]	CVPR'24	$75.65_{\pm 0.14}$	$5.73_{\pm 0.44}$	$71.01_{\pm 0.45}$	$6.83_{\pm 0.44}$	$87.06_{\pm 0.25}$	$6.22_{\pm 0.39}$	$81.45_{\pm 0.68}$	$5.35_{\pm 0.52}$	-	-
EASE [26]	CVPR'24	76.17	7.82	73.27	8.51	87.76	5.94	78.89^{*}	7.89	$49.70^{\ddagger}$	-
CPrompt [44]	CVPR'24	$77.14_{\pm 0.11}$	$5.97_{\pm 0.68}$	$74.79_{\pm 0.28}$	$7.34_{\pm 0.65}$	$87.82_{\pm 0.21}$	$5.06_{\pm 0.50}$	$82.97_{\pm 0.34}$	$7.45_{\pm 0.93}$	$51.07_{\pm 0.44}^{*}$	$9.82_{\pm 0.62}$
VPT-NSP^2 [49]	NeurIPS'24	$78.88_{\pm 0.50}$	$5.06_{\pm 0.26}$	$76.13_{\pm 0.57}$	$5.10_{\pm 0.37}$	$\underline{91.74}_{\pm 0.63}$	$3.28_{\pm 0.45}$	$83.54_{\pm 0.77}$	$8.54_{\pm 0.48}$	$50.20_{\pm 0.75}^{*}$	$10.23_{\pm 0.77}$
VPT-CPG [22]	AAAI'25	$78.63_{\pm 0.52}$	$7.18_{\pm 0.62}$	$75.33_{\pm 0.49}^{*}$	$7.08_{\pm 0.52}$	$90.63_{\pm 0.44}$	$3.98_{\pm 0.65}$	$83.21_{\pm 0.67}$	$7.09_{\pm 0.82}$	$45.13_{\pm 0.65}^{*}$	$12.86_{\pm 0.70}$
CAPrompt [23]	AAAI'25	$72.17_{\pm 0.44}^{*}$	$5.42_{\pm 0.47}$	$69.03_{\pm 0.68}^{*}$	$6.63_{\pm 0.76}$	$86.27_{\pm 0.42}^{*}$	$4.96_{\pm 0.37}$	$74.77_{\pm 0.80}^{*}$	$5.21_{\pm 0.51}$	$45.83_{\pm 0.86}^{*}$	$13.65_{\pm 0.83}$
SD-LoRA [32]	ICLR'25	$77.34_{\pm 0.35}$	$5.85_{\pm 0.24}$	$75.26_{\pm 0.37}$	$6.24_{\pm 0.28}$	$88.01_{\pm 0.31}$	$4.93_{\pm 0.22}$	$81.01_{\pm 0.42}^{*}$	$7.03_{\pm 0.35}$	$55.85_{\pm 0.68}^{*}$	$13.11_{\pm 0.52}$
CPrompt-KAC [51]	CVPR'25	78.07	5.55	75.73	6.04	87.19	5.22	80.46^{*}	7.53	52.70^{*}	13.42
DIA [29]	CVPR'25	79.03	5.20	76.32	5.90	90.80	4.05	82.81^{*}	6.80	54.29^{*}	12.75
BiLoRA [34]	CVPR'25	$77.95_{\pm 0.14}$	$5.37_{\pm 0.28}$	$72.41_{\pm 0.76}$	$7.58_{\pm 0.42}$	$87.46_{\pm 0.76}$	$5.07_{\pm 0.31}$	$76.56_{\pm 0.41}$	$8.24_{\pm 0.45}$	$50.66_{\pm 0.72}^{*}$	$14.17_{\pm 0.78}$
LoRA-DRS [33]	CVPR'25	$75.94_{\pm 0.46}$	$6.22_{\pm 0.30}$	$73.78_{\pm 0.44}$	$6.85_{\pm 0.36}$	$89.14_{\pm 0.23}$	$4.48_{\pm 0.24}$	$75.42_{\pm 0.55}^{*}$	$8.65_{\pm 0.48}$	$\underline{59.63}_{\pm 0.62}^{*}$	$12.52_{\pm 0.55}$
SEMA [27]	CVPR'25	78.00	5.73	74.53	6.52	86.98	5.35	80.92^{*}	7.29	53.32	13.29
CL-LoRA [38]	CVPR'25	78.48^{*}	5.19	76.17^{*}	5.95	87.73^{*}	5.09	82.11^{*}	7.17	52.33^{*}	13.63
MoAL [30]	CVPR'25	$\underline{79.33}$	4.95	$\underline{76.85}$	5.76	90.49	4.23	$\underline{83.95}^{*}$	6.82	52.01	12.80
Seq-FT	Baseline	$74.47_{\pm 0.52}$	$17.99_{\pm 0.64}$	$71.95_{\pm 0.47}$	$21.19_{\pm 0.73}$	$85.63_{\pm 0.36}$	$13.18_{\pm 0.55}$	$73.52_{\pm 0.41}$	$24.41_{\pm 0.70}$	$45.20_{\pm 0.98}$	$29.92_{\pm 0.72}$
Our Method	This work	$\mathbf{82.40}_{\pm 0.21}$	$4.49_{\pm 0.50}$	$\mathbf{80.92}_{\pm 0.18}$	$5.61_{\pm 0.41}$	$\mathbf{92.53}_{\pm 0.09}$	$3.73_{\pm 0.31}$	$\mathbf{86.37}_{\pm 0.40}$	$6.01_{\pm 0.62}$	$\mathbf{64.27}_{\pm 0.77}$	$11.35_{\pm 0.61}$
Joint Upper Bound		$85.95_{\pm 0.32}$	-	$85.95_{\pm 0.32}$	-	$93.86_{\pm 0.22}$	-	$90.38_{\pm 0.28}$	-	$67.33_{\pm 0.35}$	-
Oracle Upper Bound		$94.40_{\pm 0.25}$	-	$96.28_{\pm 0.28}$	-	$99.18_{\pm 0.17}$	-	$97.63_{\pm 0.19}$	-	$92.90_{\pm 0.21}$	-

†, ‡, and *: accuracies reproduced from [44], [30], and our implementation, respectively; "-": officially unavailable or not applicable results.

sequentially trained up to the final task for inference without any anti-forgetting mechanism. The results under the conventional class-incremental scenario are summarized in TABLE I. (1) Relative to the baseline, our approach provides significant improvements on all benchmarks, increasing average accuracy by 11.1% across the five benchmarks and reducing forgetting by 15.1%. (2) Compared with existing approaches, our method achieves an average accuracy improvement of 3.0% over previous state of the arts across the five benchmarks. In particular, it attains new state-of-the-art accuracy on the highly competitive 10-split ImageNet-R benchmark with a substantial margin of 3.1%. On the more challenging ImageNet-A and the cross-domain DomainNet, it surpasses the best previous results by 4.6% and 2.4%, respectively. (3) Our method shows clear advantages over existing state-of-the-art task arithmetic-based methods (LAE [61] and MoAL [30]), achieving on average 4.8% higher accuracy than MoAL. (4) Despite the absence of historical data, our method remains close to the joint-training upper bound, with an average gap of only 3.4% across all benchmarks.

We further evaluate under the long-sequence CL scenario, where the number of tasks increases to 50 or 100. As shown in TABLE II, our approach achieves an accuracy gain of 32.4% and a reduction in forgetting of 31.7% compared to the baseline. Moreover, it consistently exhibits substantially better performance than existing state-of-the-art methods, with an average accuracy advantage of 4.5% on the 50-split benchmarks and 12.6% on the 100-split benchmarks. It is worth noting that, on ImageNet-R and CIFAR-100, our method exhibits only a 2.4% and 0.8% drop in accuracy under the 50-split setting compared with its performance under the 10-split setting. Such accuracies still match or surpass those of existing methods under the 10-task scenario, *e.g.*, ours 79.98% on 50S-ImageNet-R versus 79.33% of MoAL on 10S-ImageNet-R, and ours 91.71% on 50S-CIFAR-100 versus 91.74% of VPT-NSP^2 on 10S-CIFAR-100. This demonstrates the effectiveness of our approach under long-sequence tasks. In addition, we evaluate the performance of our method and six competing approaches under task sequences ranging from 5 to 100 tasks. As shown in Fig. 3, our method shows a minor accuracy drop and substantially outperforms existing approaches, with the performance gap widening as the task sequence becomes longer.

C. Ablation Studies

We conduct an ablation study to investigate the effects of three proposed components in our framework: the backbone-level unified adaptation matrix ($\mathbf{\Theta}_t^{b_l}$) in Section IV-D, the classifier-level unified adaptation matrix ($\mathbf{\Theta}_t^{h}$) in Section IV-E,

TABLE II
PERFORMANCE COMPARISON OF THE PROPOSED METHOD WITH EXISTING APPROACHES ON LONG-SEQUENCE CONTINUAL LEARNING BENCHMARKS.

Method	Venue	50S-ImageNet-R		50S-CIFAR-100		50S-DomainNet		100S-ImageNet-R		100S-DomainNet	
		Acc.↑	Forgetting↓	Acc.↑	Forgetting↓	Acc.↑	Forgetting↓	Acc.↑	Forgetting↓	Acc.↑	Forgetting↓
L2P [18]	CVPR'22	$48.53_{\pm 0.51}$	$12.99_{\pm 0.36}$	$76.19_{\pm 0.72}$	$12.06_{\pm 0.57}$	$59.45_{\pm 0.75}$	$11.53_{\pm 0.60}$	$38.87_{\pm 0.65}$	$15.26_{\pm 0.64}$	$50.52_{\pm 0.44}$	$17.66_{\pm 0.67}$
EvoPrompt [21]	AAAI'24	$68.53_{\pm 0.66}$	$10.03_{\pm 0.45}$	$76.60_{\pm 0.87}$	$13.86_{\pm 0.53}$	$67.68_{\pm 0.77}$	$10.41_{\pm 0.68}$	$61.84_{\pm 0.71}$	$15.84_{\pm 0.74}$	$56.35_{\pm 0.58}$	$21.39_{\pm 0.76}$
OVOR-Deep [42]	ICLR'24	$60.08_{\pm 0.68}$	$5.86_{\pm 0.60}$	$65.69_{\pm 0.74}$	$14.28_{\pm 0.63}$	$66.27_{\pm 0.71}$	$7.43_{\pm 0.61}$	$40.49_{\pm 0.80}$	$8.12_{\pm 0.66}$	$47.65_{\pm 0.78}$	$8.91_{\pm 0.79}$
InfLoRA [31]	CVPR'24	$59.02_{\pm 0.47}$	$11.02_{\pm 0.32}$	$61.49_{\pm 0.55}$	$13.68_{\pm 0.36}$	$69.96_{\pm 0.46}$	$9.51_{\pm 0.55}$	$38.16_{\pm 0.51}$	$15.11_{\pm 0.49}$	$44.32_{\pm 0.72}$	$17.85_{\pm 0.60}$
EASE [26]	CVPR'24	$68.17_{\pm 0.79}$	$7.76_{\pm 0.90}$	$74.47_{\pm 0.86}$	$9.31_{\pm 0.91}$	$61.20_{\pm 0.87}$	$10.01_{\pm 0.84}$	$47.55_{\pm 0.87}$	$8.22_{\pm 0.85}$	$33.08_{\pm 0.89}$	$32.14_{\pm 0.91}$
CPrompt [44]	CVPR'24	$68.47_{\pm 0.67}$	$8.16_{\pm 0.57}$	$74.97_{\pm 0.61}$	$7.45_{\pm 0.50}$	$67.87_{\pm 0.56}$	$9.36_{\pm 0.68}$	$56.95_{\pm 0.69}$	$10.20_{\pm 0.72}$	$53.73_{\pm 0.62}$	$12.14_{\pm 0.66}$
VPT-NSP^2 [49]	NeurIPS'24	$\underline{74.91}_{\pm 0.58}$	$6.99_{\pm 0.53}$	$86.27_{\pm 0.73}$	$7.33_{\pm 0.59}$	$\underline{75.86}_{\pm 0.62}$	$8.39_{\pm 0.61}$	$\underline{66.33}_{\pm 0.71}$	$8.17_{\pm 0.73}$	$\underline{65.33}_{\pm 0.77}$	$9.30_{\pm 0.75}$
VPT-CPG [22]	AAAI'25	$73.08_{\pm 0.62}$	$12.12_{\pm 0.55}$	$\underline{87.95}_{\pm 0.58}$	$5.42_{\pm 0.42}$	$72.27_{\pm 0.66}$	$19.93_{\pm 0.71}$	$64.63_{\pm 0.64}$	$12.18_{\pm 0.59}$	$60.81_{\pm 0.69}$	$27.43_{\pm 0.82}$
MoAL [30]	CVPR'25	$60.97_{\pm 0.57}$	$9.88_{\pm 0.54}$	$58.56_{\pm 0.63}$	$11.42_{\pm 0.60}$	$53.49_{\pm 0.61}$	$9.74_{\pm 0.58}$	$48.21_{\pm 0.66}$	$14.27_{\pm 0.62}$	$39.55_{\pm 0.70}$	$16.03_{\pm 0.65}$
Seq-FT	Baseline	$63.47_{\pm 0.68}$	$25.94_{\pm 0.80}$	$66.80_{\pm 0.72}$	$29.71_{\pm 0.85}$	$35.99_{\pm 0.75}$	$55.62_{\pm 0.88}$	$45.67_{\pm 0.70}$	$36.36_{\pm 0.83}$	$35.13_{\pm 0.73}$	$49.81_{\pm 0.86}$
Our Method	This work	$\mathbf{79.98}_{\pm 0.41}$	$6.32_{\pm 0.33}$	$\mathbf{91.71}_{\pm 0.37}$	$4.17_{\pm 0.31}$	$\mathbf{80.57}_{\pm 0.76}$	$10.82_{\pm 0.69}$	$\mathbf{76.53}_{\pm 0.40}$	$6.60_{\pm 0.34}$	$\mathbf{80.33}_{\pm 0.55}$	$10.96_{\pm 0.53}$

TABLE III
ABLATION STUDY ON THE PROPOSED COMPONENTS. $\mathbf{\Theta}_t^{b_l}$ AND $\mathbf{\Theta}_t^{h}$ DENOTE BACKBONE-LEVEL AND CLASSIFIER-LEVEL ADAPTATION PARAMETERS, RESPECTIVELY, AND FTI DENOTES FORWARD TRANSFER INITIALIZATION.

Index	Components			10S-ImageNet-R		10S-CIFAR-100		10S-DomainNet		20S-ImageNet-A		50S-ImageNet-R		50S-DomainNet		100S-DomainNet	
	$\mathbf{\Theta}_t^{b_l}$	$\mathbf{\Theta}_t^{h}$	FTI	Acc.↑	Forgetting↓	Acc.↑	Forgetting↓	Acc.↑	Forgetting↓	Acc.↑	Forgetting↓	Acc.↑	Forgetting↓	Acc.↑	Forgetting↓	Acc.↑	Forgetting↓
1				74.47	17.99	85.63	13.18	73.52	24.41	45.20	29.92	63.47	25.94	35.99	55.62	35.13	49.81
2	√			80.97	2.24	90.77	1.53	83.34	3.55	56.13	8.68	71.90	4.63	54.48	6.32	43.18	8.74
3		√		76.25	17.56	85.13	15.07	73.64	25.91	55.87	28.12	62.98	33.01	36.06	66.81	41.09	58.23
4			√	76.38	15.21	87.16	11.10	74.98	23.07	55.40	21.63	63.47	25.94	36.05	55.12	35.15	49.75
5	√	√		82.25	4.89	91.98	4.42	86.09	6.39	60.80	15.20	77.40	9.09	77.47	16.06	72.16	20.78
6	√	√	√	**82.40**	4.49	**92.53**	3.73	**86.37**	6.01	**64.27**	11.35	**79.98**	6.32	**80.57**	10.82	**80.33**	10.96

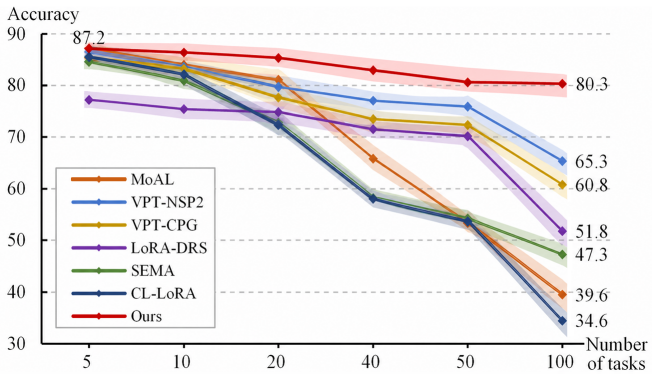


Fig. 3. Performance comparison on the DomainNet benchmark with 200 classes split into 5∼100 tasks. Six state-of-the-art methods are compared with our approach: VPT-NSP^2 [49] (NeurIPS'24), VPT-CPG [22] (AAAI'25), MoAL [30] (CVPR'25), LoRA-DRS [33] (CVPR'25), SEMA [27] (CVPR'25), and CL-LoRA [38] (CVPR'25). Our method exhibits only minor degradation in accuracy as the number of tasks increases, surpassing all compared methods by more than 15% in the 100-task scenario.

and the forward transfer initialization (FTI) strategy in Section IV-F. The quantitative results are reported in TABLE III. We first observe that the sequential fine-tuning baseline (Index 1) suffers from severe catastrophic forgetting. By incorporating $\mathbf{\Theta}_t^{b_l}$ (Index 1→2), the average accuracy across all benchmarks achieves a significant gain of 6.8%, and the forgetting is reduced by a margin of 13.5%. This demonstrates that backbone adaptation plays a dominant role in anti-forgetting. In contrast, using $\mathbf{\Theta}_t^{h}$ or FTI alone yields only limited improvements (1→3, 1→4), indicating that neither of them can effectively prevent forgetting in the absence of unified backbone optimization.

TABLE IV
COMPARISON OF DIFFERENT CLASSIFIER SOLUTION VARIANTS.

Variants	10S-ImageNet-R		10S-DomainNet		50S-ImageNet-R		50S-DomainNet	
	Acc.	Forgetting	Acc.	Forgetting	Acc.	Forgetting	Acc.	Forgetting
a.	79.63	9.68	80.72	15.37	75.28	12.04	69.95	26.27
b.	80.25	6.60	83.04	9.74	77.38	7.56	78.73	12.43
c.	78.80	3.86	76.22	1.43	76.30	5.88	59.22	7.45
d.	72.03	0.33	74.65	0.58	64.55	0.85	48.37	2.59
Ours	**82.40**	4.49	**86.37**	6.01	**79.98**	6.32	**80.57**	10.82

When combining $\mathbf{\Theta}_t^{b_l}$ with $\mathbf{\Theta}_t^{h}$ (Index 5), we observe further improvements across all benchmarks compared to using $\mathbf{\Theta}_t^{b_l}$ alone (Index 2→5). The effect of $\mathbf{\Theta}_t^{h}$ becomes increasingly important in the long-sequence scenario, with an average accuracy gain of 4.9% on long-sequence benchmarks (50S/100S), as opposed to 2.4% on conventional benchmarks (10S/20S). This suggests that $\mathbf{\Theta}_t^{h}$ is essential for correcting and aligning the classifier against cross-task confusion, a problem that naturally intensifies as the number of tasks increases. Notably, even without FTI, our approach (Index 5) already achieves state-of-the-art performance across all benchmarks. Introducing FTI (Index 5→6) brings additional benefits, particularly on long-sequence benchmarks, indicating it enhances forward knowledge transfer and results in more stable performance. Finally, our full method (Index 6) integrated all three components achieves the best overall performance.

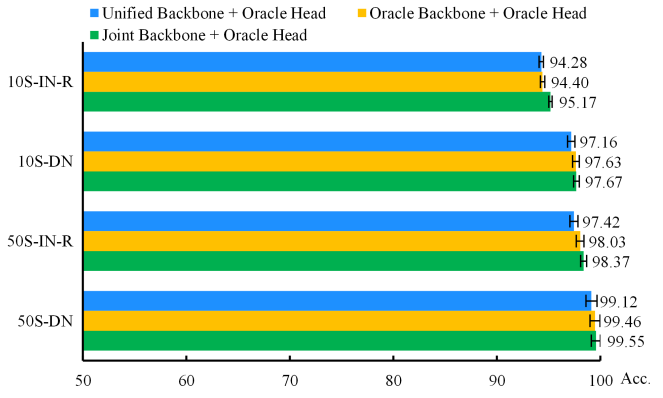


Fig. 4. Accuracy comparison of different model configurations. "Unified Backbone + Oracle Head", "Oracle Backbone + Oracle Head", and "Joint Backbone + Oracle Head" represent settings that employ our backbone, task-specific backbones, and a jointly trained backbone, respectively, each combined with task-specific classifiers.

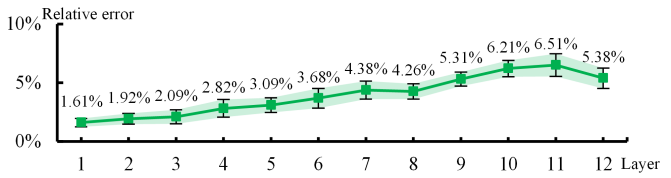


Fig. 5. Relative optimization error across different ViT blocks on the 10-split ImageNet-R benchmark.

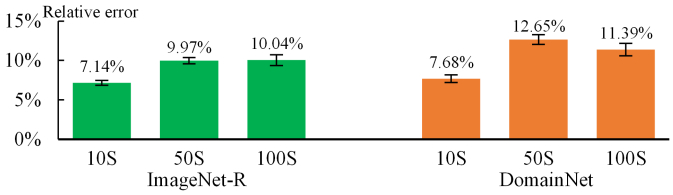


Fig. 6. Representation error of the class token at the final ViT block under different task numbers (10, 50, and 100) on the ImageNet-R and DomainNet benchmarks.

To assess the benefit of our derived unified optimization solution for classifier proposed in Section IV-E, we design four variants and compare them with our method, as shown in TABLE IV. They only differ in how target logits are approximated in Eq. (25). Variant *a.* sets $\alpha = 0$ and $\mathbf{Z}_{i,j} = \hat{\mathbf{H}}_i = \mathbf{X}_i^h \mathbf{W}_i^h$ for $j = i$, which is exactly equivalent to the existing optimization solution for the classifier [30], [63]–[65]. Compared with this restricted solution, our generalized formulation achieves an average accuracy improvement of 5.94%, demonstrating its rationality in the optimization of the classifier. Variant *b.* sets $\alpha = 0$ and $\beta = 200$. Relative to Variant *a.*, the within-task logits are magnified to enforce stronger class separation, leading to an accuracy gain of 3.46% and a forgetting reduction of 6.76%, which highlights the effectiveness of introducing within-task correction. Variant *c.* sets $\alpha = 1$ and $\beta = 200$, *i.e.*, $\mathbf{Z}_{i,j} = \hat{\mathbf{Z}}_{i,j} = \mathbf{X}_i^h \mathbf{W}_j^h$ for $i \neq j$, which retains the raw predictions for cross-task logits without calibration compared to Variant *b*. Its accuracy drops by 7.22% relative to Variant *b.*, with the decline being more significant on DomainNet, a cross-domain CIL benchmark where task confusion is more severe. Variant *d.* applies no correction at all, *i.e.*, $\mathbf{Z}_{i,j} = \hat{\mathbf{Z}}_{i,j}$ for both $i = j$ and $i \neq j$, thereby directly preserving the biased predictions. As a result, it exhibits the smallest forgetting but also the poorest accuracy across all benchmarks. Our method employs both reasonable cross-task logits scaling $\alpha = 0.3$ and within-task magnification $\beta = 200$, thus simultaneously correcting cross-task confusion and enhancing within-task discrimination. As a result, it consistently achieves the best accuracy, establishing its soundness and superiority in unified classifier optimization.

D. Error Analysis of Optimization and Representation

In this subsection, we analyze the gap between our method and the upper bound in accuracy, optimization and representation, and further conduct a generalization error analysis.

1) Comparison in Accuracy: To quantify the accuracy gap caused by representation discrepancies, we compare three inference configurations: Global/Oracle/Joint Backbone + Oracle Head. The "Unified" backbone uses our method to compute the unified adaptation parameters ($\mathbf{\Theta}_t^{b_l *}$) for inference (Eq. (4)); the "Oracle" backbone/head selects the task-specific LoRA or classifier according to the ground-truth task identifier during inference (Eq. (3)); and the "Joint" backbone extracts features using the backbone jointly trained with Eq. (1). The comparison of these configurations on 10-split ImageNet-R is presented in Fig. 4. The "Unified Backbone + Oracle Head" setting employs the representations produced by our method but uses the task-specific classifier during inference. Its accuracy is close to that of "Oracle Backbone + Oracle Head" and "Joint Backbone + Oracle Head", with differences of only 0.38% and 0.70%, respectively. This demonstrates that the representation error produced by our method has a minimal effect on accuracy, and verifies that our method is capable of learning representations that approach those of the upper-bound paradigms.

2) Error Analysis with Respect to Oracle Upper Bound: In this part, we conduct an in-depth analysis of the optimization and representation errors of our method relative to the *oracle upper bound*. In terms of the optimization error, we compute the objective in Eq. (5) for each ViT block across all tasks on 10-split ImageNet-R. For an intuitive comparison, we report the *relative error* $\left\|\mathbf{X}_i^{b_l}(\mathbf{W}^{b_l} + \mathbf{\Theta}_t^{b_l *}) - \mathbf{Y}_i^{b_l}\right\|_{\mathrm{F}}^2 / \left\|\mathbf{Y}_i^{b_l}\right\|_{\mathrm{F}}^2$, as shown in Fig. 5. We observe that the error increases as the network depth grows, which can be attributed to two main factors: 1) errors from earlier layers may propagate to subsequent layers, and 2) features in deeper blocks become increasingly task- and category-specific, which amplifies cross-task discrepancies and raises the optimization difficulty. At the final layer, the optimization error reaches approximately 5%.

We further evaluate the representation discrepancy between our method and the oracle upper bound by the cosine distance of the class token at the final block. The results under different numbers of tasks (10, 50, and 100) are presented in Fig. 6. Although the representation error slightly increases from 10 to 50 tasks (2.83% on ImageNet-R and 4.97% on DomainNet), the corresponding accuracy gaps remain minor: as shown in Fig. 4, the accuracy gap between "Unified Backbone + Oracle

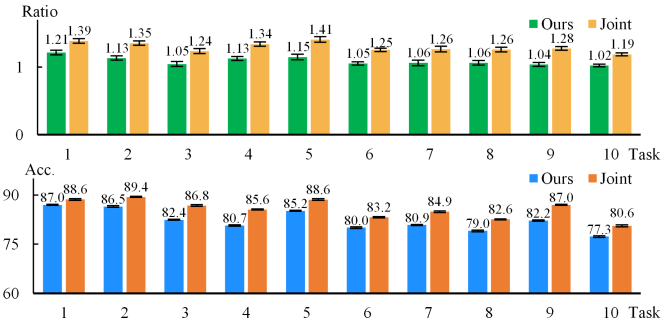


Fig. 7. Comparison of class separability and accuracy between the proposed method and joint training on the 10-split ImageNet-R benchmark. Class separability is quantified as the ratio of inter-class to intra-class distances, (*i.e.*, $d_{\text{inter}}/d_{\text{intra}}$), where a larger ratio indicates better discriminability.

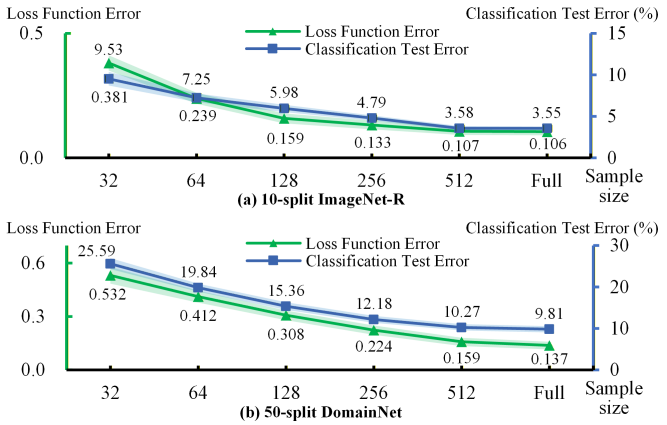


Fig. 8. Classification test error *vs.* loss function error under different numbers of training samples used to compute the unified solution.

Head" and "Oracle Backbone + Oracle Head" only rises from 0.12% (on 10S-ImageNet-R) to 0.61% (on 50S-ImageNet-R), and even decreases from 0.47% (on 10S-DomainNet) to 0.34% (on 50S-DomainNet). These observations indicate that the increased representation discrepancy has a negligible impact on accuracy. Furthermore, the representation errors show minor variation under long-sequence settings (50 and 100 tasks), confirming the robustness of our method in continual learning with long task sequences.

3) Error Analysis with Respect to Joint Upper Bound: In this part, we analyze the difference between the representations extracted by our method and the joint training paradigm. Concretely, we quantify the separability across class-wise representations by measuring the intra-class and inter-class distances. The intra-class distance (denoted as d_{intra}) is defined as the average distance between the representations of all samples within a class and the corresponding class prototype (*i.e.*, the mean representation of that class). The inter-class distance (d_{inter}) is defined as the average pairwise distance between the prototypes of all classes. We characterize the overall class separability using the *ratio* of inter-class to intra-class distance: *ratio*=$d_{\text{inter}}/d_{\text{intra}}$, where a larger value indicates better discriminability. Fig. 7 presents the class separability and corresponding accuracy for our method and joint training across the 10-split ImageNet-R tasks. On average, our method

TABLE V
PERFORMANCE COMPARISON OF THE PROPOSED METHOD UNDER THE DINO-1K AND IBOT-1K PRE-TRAINING PARADIGMS.

Pre-training	Method	10S-ImageNet-R		50S-DomainNet	
		Acc.	Forgetting	Acc.	Forgetting
DINO-1k	CODA-Prompt [20]	64.02	7.70	49.86	10.52
	EASE [26]	65.97	9.24	<u>56.55</u>	15.86
	CPrompt [44]	64.66	9.26	52.10	14.03
	CL-LoRA [38]	71.34	7.05	44.84	18.87
	SEMA [27]	63.80	10.11	38.53	17.36
	VPT-NSP^2 [49]	69.42	7.15	48.36	13.75
	VPT-CPG [22]	68.32	2.78	39.46	17.48
	MoAL [30]	<u>71.83</u>	5.92	46.39	16.68
	Seq-FT	57.90	18.26	18.85	67.24
	Our Method	**74.02**	7.94	**62.41**	15.13
iBOT-1k	CODA-Prompt [20]	68.02	6.23	50.35	9.74
	EASE [26]	69.91	9.22	54.23	11.28
	CPrompt [44]	69.14	8.82	<u>56.72</u>	10.96
	CL-LoRA [38]	<u>74.94</u>	6.30	42.36	15.45
	SEMA [27]	69.65	8.95	33.94	17.72
	VPT-NSP^2 [49]	73.90	5.67	51.58	13.54
	VPT-CPG [22]	72.16	5.80	50.17	10.89
	MoAL [30]	74.77	6.56	48.29	15.37
	Seq-FT	65.33	16.82	21.23	70.59
	Our Method	**78.48**	6.83	**67.17**	14.74

achieves a separability ratio that is 0.21 lower than that of joint training, resulting in an accuracy drop of 3.6%. This small gap demonstrates that our method maintains good representation capability for continual learning.

4) Generalization Error Analysis: Beyond the loss function error, we further examine the generalization error by investigating whether reducing the proposed objective leads to lower classification test error in practice. Theoretically, this generalization ability is supported by two aspects of our framework: 1) The proposed formulation is derived from approximating joint training over all tasks, which is widely regarded as an upper bound in CL and is known to exhibit generalization. 2) The regularization term in Eq. (5) constrains the complexity of the unified adaptation parameters, thereby improving generalization through complexity control. To empirically examine the relationship between optimization and generalization, we vary the number of training samples (denoted as n) used to compute the unified solution, with $n \in \{32, 64, 128, 256, 512, \text{full training set size}\}$, and then evaluate the loss function error on the full training set (measured by Eq. (5)) and the classification test error on the full test set (measured by the accuracy gap to the upper bound). As shown in Fig. 8, when the number of samples increases, the loss function error consistently decreases, and the classification test error follows a similar downward trend on both benchmarks. This observation indicates a strong empirical correlation between minimizing the proposed objective and reducing generalization error, suggesting that our unified optimization framework not only fits the input-output features well but also yields favorable generalization performance.

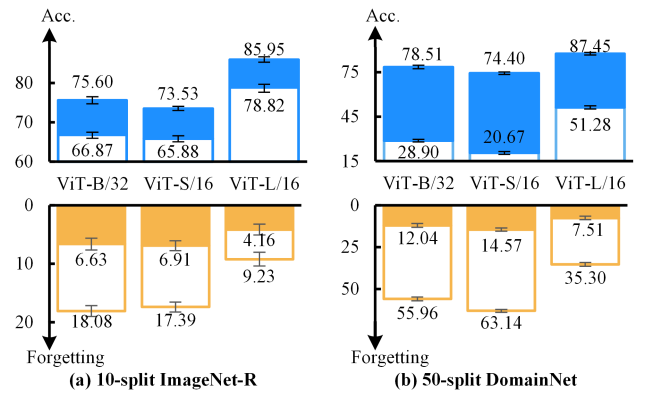


Fig. 9. Comparison between the proposed method and the Seq-FT baseline across different ViT architectures (ViT-Base/32, ViT-Small/16, and ViT-Large/16) pre-trained on ImageNet-21K. Annotated values on the solid bars indicate the accuracy or forgetting of the proposed method, while those on the hollow bars correspond to the Seq-FT baseline.

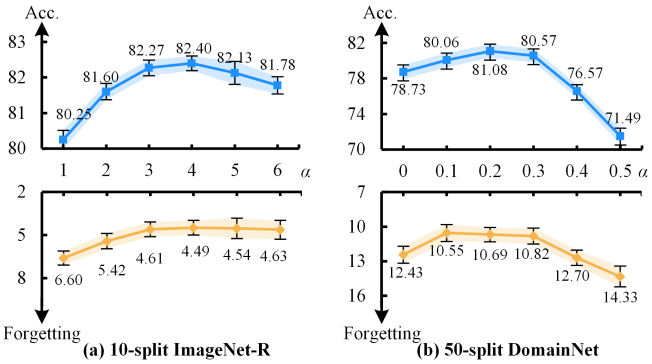


Fig. 10. Accuracy and forgetting curves with respect to the logits scaling factor α ranging from 0 to 0.5, with the default value set to 0.3.

E. Generalization Across Pre-training Paradigms and Model Capacities

To assess the generalization ability of our framework, we further evaluate its performance under different pre-training paradigms and model capacities. TABLE V reports the results with two self-supervised pre-training paradigms on the ImageNet-1k dataset: DINO-1k [80] and iBOT-1k [81]. Compared to the sequential fine-tuning baseline, our approach improves accuracy by 29.8% and reduces forgetting by 31.2% under DINO-1k, and by 29.6% and 32.9% under iBOT-1k, respectively. Moreover, our method outperforms recent state-of-the-art CL approaches, achieving higher accuracy by an average of 2.9% on 10-split ImageNet-R and 8.2% on 50-split DomainNet benchmarks. It demonstrates that the proposed approach is not restricted to a single pre-training strategy and remains effective under different paradigms.

We additionally apply our method to various backbone architectures to investigate the influence of model capacity, including ViT-Base/32, ViT-Small/16 and ViT-Large/16, with all initialized from ImageNet-21K pre-trained weights. As shown in Fig. 9, our method achieves an average accuracy improvement of 21.2% and a forgetting reduction of 24.6% compared to the baseline. The performance consistently improves across these architectures in both conventional and long-sequence CL settings, confirming the scalability of our method in terms of model capacity.

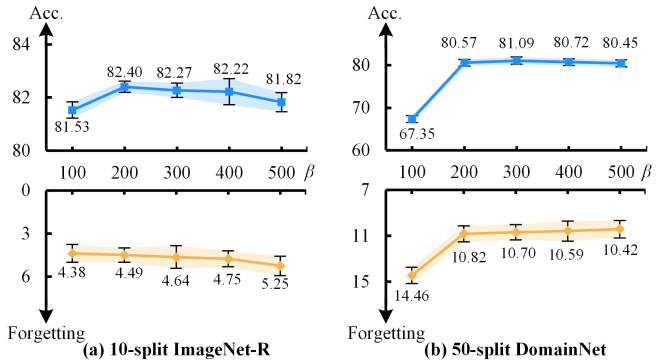


Fig. 11. Accuracy and forgetting curves *w.r.t.* the magnification factor β ranging from 100 to 500, with the default value set to 200.

F. Impacts of Hyper-parameters

In this subsection, we investigate the impact of five key hyper-parameters: the *logits scaling factor* α and the *magnification factor* β introduced in Eq. (25), the *regularization coefficient* λ in Eq. (5) and Eq. (18), the *LoRA insertion depth* L, *i.e.*, the number of backbone layers equipped with LoRA modules for fine-tuning, and the *LoRA rank* r.

1) Logits Scaling Factor α: The hyper-parameter α controls the contribution of non-target classifiers in the scaled-logit approximation (Eq. (25a)). An excessively large α may lead non-target logits to exceed those of the correct class, thereby aggravating cross-task confusion. In contrast, an excessively small α enlarges the optimization difficulty of the objective (Eq. (18)) and results in higher fitting errors. Therefore, we restrict α to the range $[0, 0.5]$, and the corresponding accuracy and forgetting curves are presented in Fig. 10. We observe that accuracy first increases and then decreases as α grows from zero, and the decline becomes amplified in long-sequence scenarios where cross-task confusion is more severe. Moderate values of α in the range of 0.1$\sim$0.3 provide a favorable balance. Considering the overall performance across all benchmarks, we choose 0.3 as the default value for α. Under the settings $\alpha = 0.3$ and $\alpha = 0$, the corresponding average values of the objective Eq. (18) are 2.47 and 2.95 on 10S-ImageNet-R, respectively (with 2.47$<$2.95). This confirms that the proposed logit scaling strategy effectively reduces the fitting error, thereby enhancing CL performance.

2) Magnification Factor β: The hyper-parameter β serves as a margin-enforcing coefficient for within-task logits (Eq. (25b)). By amplifying the correct class logits relative to incorrect classes, β enhances within-task discrimination and mitigates within-task errors. Since the logits of the correct class are typically on the order of hundreds, we restrict β to the range of $[100, 500]$. As shown in Fig. 11, the accuracy rises and tends to stabilize as β increases. Further enlarging β leads to marginal performance gains and may introduce numerical instability. We observe that setting $\beta = 200$ provides sufficient separation from non-target logits, enabling reliable within-task classification without compromising optimization stability.

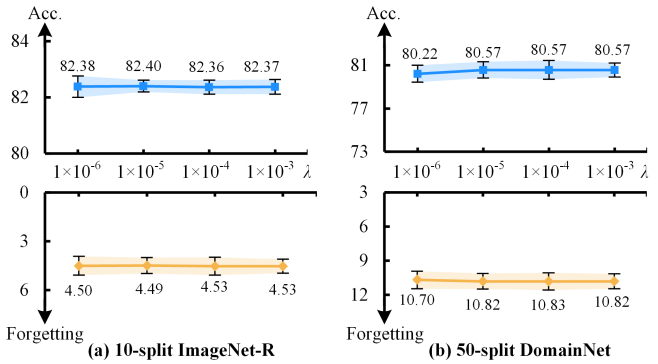


Fig. 12. Accuracy and forgetting curves *w.r.t.* the regularization coefficient λ ranging from 1×10^{-6} to 1×10^{-3}, with the default value set to 1×10^{-5}.

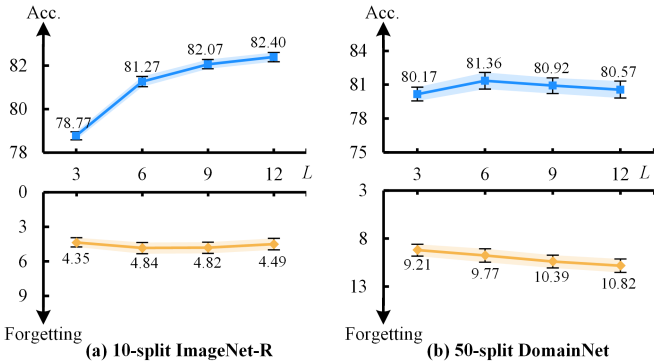


Fig. 13. Accuracy and forgetting curves *w.r.t.* the number of layers equipped with LoRA modules (denoted as L). LoRAs are inserted into the first {3, 6, 9, 12} layers, with the default configuration using all 12 layers.

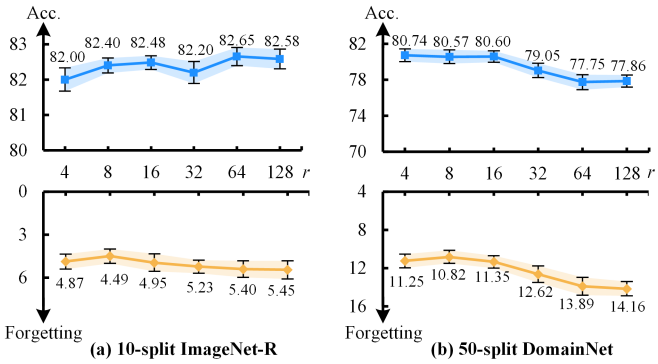


Fig. 14. Accuracy curves *w.r.t.* the LoRA rank r varied from 4 to 128, with the default value set to 8.

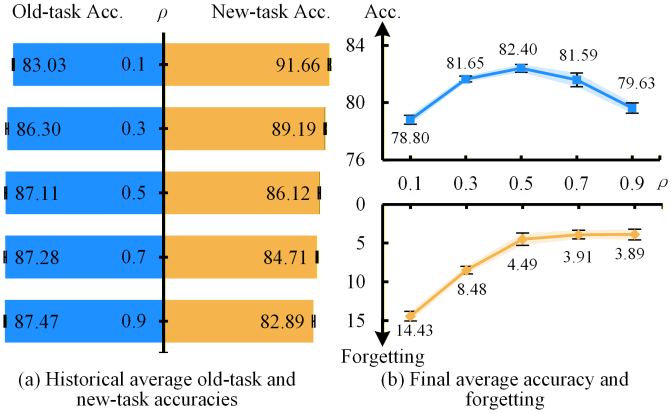


Fig. 15. Effect of the stability-plasticity balancing coefficient $\rho \in [0.1, 0.9]$ on the 10-split ImageNet-R benchmark, with the default value set to 0.5.

3) Regularization Coefficient λ: The effect of the regularization coefficient λ used in Eq. (5) and Eq. (18) is illustrated in Fig. 12. Across the range from 1×10^{-6} to 1×10^{-3}, accuracy and forgetting exhibit only negligible variations on both benchmarks. It suggests that our method is insensitive to the choice of λ and able to maintain stable performance without careful tuning of this hyper-parameter.

4) LoRA Insertion Depth: We insert LoRA modules into the first L ViT blocks ($L \in \{3, 6, 9, 12\}$), with corresponding performance illustrated in Fig. 13. In the conventional CL scenario (10-split ImageNet-R), increasing L enhances the model's plasticity, leading to higher accuracy when more blocks are fine-tuned. In contrast, in the long-sequence CL scenario (50-split DomainNet), increasing L gradually raises forgetting, and accuracy shows a slight decline. This is because stability becomes dominant in long sequences, where reducing the number of fine-tuned blocks helps retain more stability. Nevertheless, to avoid excessive hyper-parameter tuning for specific scenarios or datasets, we adopt a unified default configuration that inserts LoRA modules into all the 12 blocks.

5) LoRA Rank r: The rank r in LoRA modules controls the capacity of the backbone. We vary r from 4 to 128 to analyze its impact. As shown in Fig. 14, our method maintains stable performance across a wide range of ranks, indicating that it is relatively robust to the choice of r. In the long-sequence setting of 50-split DomainNet, excessively increasing r leads to a slight performance decrease, since a larger number of trainable parameters may introduce stronger forgetting. Overall, a moderate rank such as $r = 8$ or $r = 16$ achieves a favorable trade-off between performance and efficiency, which is consistent with our default setting.

G. Stability-Plasticity Trade-Off

Different continual learning scenarios may require different balances between stability and plasticity. Such controllability can be achieved by additionally introducing a weighting coefficient ρ into the progressive update of the cumulative statistics in Eq. (10): $\mathbf{F}_t^{(\rho)} = 2\rho\mathbf{F}_{t-1} + 2(1-\rho)\mathbf{X}_t^\top\mathbf{X}_t$, $\mathbf{G}_t^{(\rho)} = 2\rho\mathbf{G}_{t-1} + 2(1-\rho)\mathbf{X}_t^\top(\mathbf{Y}_t - \mathbf{X}_t\mathbf{W})$, where $\rho = 0.5$ denotes our default unbiased setting. A larger ρ improves stability, while a smaller ρ favors plasticity. We vary ρ from 0.1 to 0.9 on 10-split ImageNet-R to examine this controllability, and report the historical average accuracy (*i.e.*, the accuracy averaged over all task transitions) on old and new tasks to assess stability and plasticity. As shown in Fig. 15, increasing ρ improves the historical average old-task accuracy from 83.03% to 87.47%, while the new-task accuracy decreases from 91.66% to 82.89%, demonstrating a controllable shift from plasticity to stability. Meanwhile, the final average forgetting decreases from 14.43% to 3.89%, while the final average accuracy peaks at 82.40% when $\rho = 0.5$. This confirms that our framework enables controllable stability-plasticity adjustment, and $\rho = 0.5$ offers a balanced trade-off.

TABLE VI
COMPUTATION AND STORAGE OVERHEADS ON THE 20-SPLIT IMAGENET-R BENCHMARK.

Method	Training stage		Inference stage		
	Extra memory (MB)	Training time (ms/image)	Inference GFLOPs	Inference time (ms/image)	Acc.
CPrompt [44]	3.68	7.21	73.98	2.48	74.79
VPT-NSP^2 [49]	56.77	2.15	35.45	0.85	76.13
VPT-CPG [22]	56.75	2.22	35.46	0.87	75.33
Seq-FT	0	2.02	35.16	0.37	71.95
Our method	111.96	2.03	35.16	0.37	**80.92**

H. Computation and Storage Overheads

We evaluate the computation and storage overheads of the proposed method in comparison with the baseline and three representative methods in the following aspects: *1) Extra memory*: the additionally occupied memory compared to the sequential fine-tuning baseline. Our method requires additional storage for the following variables: $\mathbf{F}_t^{\{b_l,h\}} \in \mathbb{R}^{D\times D}$, $\mathbf{G}_t^{b_l} \in \mathbb{R}^{D\times E}$, $\mathbf{P}_t^{h_{(t)}} \in \mathbb{R}^{D\times C_{(t)}}$, $\dot{\mathbf{A}}_t^{b_l} \in \mathbb{R}^{D\times r}$ and $\dot{\mathbf{B}}_t^{b_l} \in \mathbb{R}^{r\times E}$. Note that $\mathbf{Q}_t^{h_{(t)}}$ does not incur extra memory overhead, as it can be computed on the fly. Substituting $D=768$, $E=2304$, $r=8$ for $L=12$ layers and $C_{(t)}=200$ for ImageNet-R, the memory of these variables amounts to 111.96 MB. *2) Training and inference time*: we uniformly measure all compared methods using a single NVIDIA GeForce RTX 4090 GPU. The training time is defined as the average time in milliseconds (ms) required to process one image, including forward propagation, backward propagation and parameter updates. For our method, it also includes the time for solving the unified adaptation parameters for each task. The inference time is measured by the forward propagation time only. *3) GFLOPs*: the number of Giga floating-point operations during inference, reflecting the computation complexity of the model.

The results on the 20-split ImageNet-R benchmark are summarized in TABLE VI. Although our method introduces a moderate increase in memory overhead compared to existing approaches (twice that of VPT-NSP^2), the substantial accuracy improvements make this trade-off acceptable. Moreover, our method exhibits competitive efficiency in terms of computation overhead, training and inference speed, which are close to those of sequential fine-tuning. These results demonstrate that our method achieves a favorable balance between accuracy and efficiency, making it a promising solution for practical deployment in large-scale continual learning tasks.

VI. CONCLUSION

In this work, we have proposed a unified optimization framework for LoRA-based continual learning, which parameterizes all task adaptations with a single unified adaptation module to avoid task identification and input-output mapping inconsistency issues. By reformulating the CL paradigm as a consistent feature mapping problem and solving a convex least-squares objective, our method enables knowledge preservation via progressive updates of cumulative feature statistics. We further extend this principle to the classifier by introducing additional cumulative statistics and tailored update rules, thereby achieving approximate joint training of both the backbone and classifier within an integrated framework. Extensive experiments demonstrate that our method attains new state-of-the-art performance across multiple benchmarks, with particular strength in challenging long-sequence scenarios. Future work will focus on extending this framework to broader continual learning scenarios and applying it to large-scale real-world domains such as robotics and autonomous driving systems.

REFERENCES

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is All you Need," in *Neural Inf. Process. Syst.*, 2017, pp. 5998–6008.

[2] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale," in *Int. Conf. Learn. Represent.*, 2021, pp. 611–631.

[3] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, "End-to-End Object Detection with Transformers," in *Eur. Conf. Comput. Vis.*, vol. 12346, 2020, pp. 213–229.

[4] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever, "Learning Transferable Visual Models From Natural Language Supervision," in *Int. Conf. Mach. Learn.*, vol. 139, 2021, pp. 8748–8763.

[5] A. Kirillov, E. Mintun, N. Ravi, H. Mao, C. Rolland, L. Gustafson, T. Xiao, S. Whitehead, A. C. Berg, W.-Y. Lo, P. Dollár, and R. B. Girshick, "Segment Anything," in *Int. Conf. Comput. Vis.*, 2023, pp. 3992–4003.

[6] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-Rank Adaptation of Large Language Models," in *Int. Conf. Learn. Represent.*, 2022, pp. 12 513–12 525.

[7] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient Finetuning of Quantized LLMs," in *Neural Inf. Process. Syst.*, 2023, pp. 10 088–10 115.

[8] E. J. Wang, "Alpaca-LoRA," https://github.com/tloen/alpaca-lora, Oct. 2025.

[9] N. Savage, "Learning Over a Lifetime," *Nature*, 2022, nature Outlook article, DOI: 10.1038/d41586-022-01962-y. [Online]. Available: https://www.nature.com/articles/d41586-022-01962-y

[10] S. Dohare, J. F. Hernandez-Garcia, Q. Lan, P. Rahman, A. R. Mahmood, and R. S. Sutton, "Loss of plasticity in deep continual learning," *Nature*, vol. 632, no. 8026, pp. 768–774, Aug. 2024.

[11] D.-W. Zhou, Q.-W. Wang, Z.-H. Qi, H.-J. Ye, D.-C. Zhan, and Z. Liu, "Class-Incremental Learning: A Survey," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 12, pp. 9851–9873, 2024.

[12] L. Wang, X. Zhang, H. Su, and J. Zhu, "A Comprehensive Survey of Continual Learning: Theory, Method and Application," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 8, pp. 5362–5383, 2024.

[13] M. McCloskey and N. J. Cohen, "Catastrophic interference in connectionist networks: The sequential learning problem," in *Psychology of Learning and Motivation*. Elsevier, 1989, vol. 24, pp. 109–165.

[14] R. Ratcliff, "Connectionist models of recognition memory: Constraints imposed by learning and forgetting functions." *Psychological Review*, vol. 97, no. 2, p. 285, 1990.

[15] Z. Wang, E. Yang, L. Shen, and H. Huang, "A Comprehensive Survey of Forgetting in Deep Learning Beyond Continual Learning," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 47, no. 3, pp. 1464–1483, 2025.

[16] M. Jia, L. Tang, B.-C. Chen, C. Cardie, S. J. Belongie, B. Hariharan, and S.-N. Lim, "Visual Prompt Tuning," in *Eur. Conf. Comput. Vis.*, vol. 13693, 2022, pp. 709–727.

[17] S. Chen, C. Ge, Z. Tong, J. Wang, Y. Song, J. Wang, and P. Luo, "AdaptFormer: Adapting Vision Transformers for Scalable Visual Recognition," in *Neural Inf. Process. Syst.*, 2022, pp. 16 664–16 678.

[18] Z. Wang, Z. Zhang, C.-Y. Lee, H. Zhang, R. Sun, X. Ren, G. Su, V. Perot, J. Dy, and T. Pfister, "Learning to Prompt for Continual Learning," in *Comput. Vis. Pattern Recognit.*, 2022, pp. 139–149.

[19] Z. Wang, Z. Zhang, S. Ebrahimi, R. Sun, H. Zhang, C.-Y. Lee, X. Ren, G. Su, V. Perot, J. G. Dy, and T. Pfister, "DualPrompt: Complementary Prompting for Rehearsal-Free Continual Learning," in *Eur. Conf. Comput. Vis.*, vol. 13686, 2022, pp. 631–648.

[20] J. S. Smith, L. Karlinsky, V. Gutta, P. Cascante-Bonilla, D. Kim, A. Arbelle, R. Panda, R. Feris, and Z. Kira, "CODA-Prompt: COntinual Decomposed Attention-Based Prompting for Rehearsal-Free Continual Learning," in *Comput. Vis. Pattern Recognit.*, 2023, pp. 11 909–11 919.

[21] M. R. Kurniawan, X. Song, Z. Ma, Y. He, Y. Gong, Y. Qi, and X. Wei, "Evolving Parameterized Prompt Memory for Continual Learning," in *AAAI Conf. Artif. Intell.*, 2024, pp. 13 301–13 309.

[22] Y. Lu, S. Zhang, D. Cheng, G. Liang, Y. Xing, N. Wang, and Y. Zhang, "Training Consistent Mixture-of-Experts-Based Prompt Generator for Continual Learning," in *AAAI Conf. Artif. Intell.*, 2025, pp. 19 152–19 160.

[23] Q. Li and J. Zhou, "CAPrompt: Cyclic Prompt Aggregation for Pre-Trained Model Based Class Incremental Learning," in *AAAI Conf. Artif. Intell.*, 2025, pp. 18 421–18 429.

[24] S. Zhang, Y. Lu, D. Cheng, Y. Xing, N. Wang, P. Wang, and Y. Zhang, "VPT-NSP2++: Importance-Aware Visual Prompt Tuning in Null Space for Continual Learning," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 48, no. 4, pp. 4318–4335, 2026.

[25] X. L. Li and P. Liang, "Prefix-Tuning: Optimizing Continuous Prompts for Generation," in *Annual Meeting of the Association for Computational Linguistics*, 2021, pp. 4582–4597.

[26] D.-W. Zhou, H.-L. Sun, H.-J. Ye, and D.-C. Zhan, "Expandable Subspace Ensemble for Pre-Trained Model-Based Class-Incremental Learning," in *Comput. Vis. Pattern Recognit.*, 2024, pp. 23 554–23 564.

[27] H. Wang, H. Lu, L. Yao, and D. Gong, "Self-Expansion of Pre-trained Models with Mixture of Adapters for Continual Learning," in *Comput. Vis. Pattern Recognit.*, 2025, pp. 10 087–10 098.

[28] Y. Tan, Q. Zhou, X. Xiang, K. Wang, Y. Wu, and Y. Li, "Semantically-Shifted Incremental Adapter-Tuning is A Continual ViTransformer," in *Comput. Vis. Pattern Recognit.*, 2024, pp. 23 252–23 262.

[29] J. Li, S. Wang, B. Qian, Y. He, X. Wei, Q. Wang, and Y. Gong, "Dynamic Integration of Task-Specific Adapters for Class Incremental Learning," in *Comput. Vis. Pattern Recognit.*, 2025, pp. 30 545–30 555.

[30] Z. Gao, W. Jia, X. Zhang, D. Zhou, K. Xu, D. Feng, Y. Dou, X. Mao, and H. Wang, "Knowledge Memorization and Rumination for Pre-trained Model-based Class-Incremental Learning," in *Comput. Vis. Pattern Recognit.*, 2025, pp. 20 523–20 533.

[31] Y.-S. Liang and W.-J. Li, "InfLoRA: Interference-Free Low-Rank Adaptation for Continual Learning," in *Comput. Vis. Pattern Recognit.*, 2024, pp. 23 638–23 647.

[32] Y. Wu, H. Piao, L.-K. Huang, R. Wang, W. Li, H. Pfister, D. Meng, K. Ma, and Y. Wei, "SD-LoRA: Scalable Decoupled Low-Rank Adaptation for Class Incremental Learning," in *Int. Conf. Learn. Represent.*, 2025, pp. 10 049–10 065.

[33] X. Liu and X. Chang, "LoRA Subtraction for Drift-Resistant Space in Exemplar-Free Continual Learning," in *Comput. Vis. Pattern Recognit.*, 2025, pp. 15 308–15 318.

[34] H. Zhu, Y. Zhang, J. Dong, and P. Koniusz, "BiLoRA: Almost-Orthogonal Parameter Spaces for Continual Learning," in *Comput. Vis. Pattern Recognit.*, 2025, pp. 25 613–25 622.

[35] R. Chitale, A. Vaidya, A. Kane, and A. S. Ghotkar, "Task Arithmetic with LoRA for Continual Learning," 2023, pp. 1–8.

[36] J. Yu, Y. Zhuge, L. Zhang, P. Hu, D. Wang, H. Lu, and Y. He, "Boosting Continual Learning of Vision-Language Models via Mixture-of-Experts Adapters," in *Comput. Vis. Pattern Recognit.*, 2024, pp. 23 219–23 230.

[37] G. Zhang, L. Wang, G. Kang, L. Chen, and Y. Wei, "SLCA++: Unleash the Power of Sequential Fine-tuning for Continual Learning with Pre-training," *CoRR*, vol. abs/2408.08295, pp. 1–13, 2024.

[38] J. He, Z. Duan, and F. Zhu, "CL-LoRA: Continual Low-Rank Adaptation for Rehearsal-Free Class-Incremental Learning," in *Comput. Vis. Pattern Recognit.*, 2025, pp. 30 534–30 544.

[39] D. Marczak, B. Twardowski, T. Trzcinski, and S. Cygert, "MAGMAX: Leveraging Model Merging for Seamless Continual Learning," in *Eur. Conf. Comput. Vis.*, vol. 15143, 2024, pp. 379–395.

[40] M. K. Nori and I.-M. Kim, "Task Confusion and Catastrophic Forgetting in Class-Incremental Learning: A Mathematical Framework for Discriminative and Generative Modelings," in *Neural Inf. Process. Syst.*, 2024, pp. 47 678–47 707.

[41] Y. Wang, Z. Ma, Z. Huang, Y. Wang, Z. Su, and X. Hong, "Isolation and Impartial Aggregation: A Paradigm of Incremental Learning without Interference," in *AAAI Conf. Artif. Intell.*, 2023, pp. 10 209–10 217.

[42] W.-C. Huang, C.-F. Chen, and H. Hsu, "OVOR: OnePrompt with virtual outlier regularization for rehearsal-free class-incremental learning," in *Int. Conf. Learn. Represent.*, 2024, pp. 39 355–39 374.

[43] Y.-M. Tang, Y.-X. Peng, and W.-S. Zheng, "When Prompt-based Incremental Learning Does Not Meet Strong Pretraining," in *Int. Conf. Comput. Vis.*, 2023, pp. 1706–1716.

[44] Z. Gao, J. Cen, and X. Chang, "Consistent Prompting for Rehearsal-Free Continual Learning," in *Comput. Vis. Pattern Recognit.*, 2024, pp. 28 463–28 473.

[45] Y. Kim, Y. Li, and P. Panda, "One-Stage Prompt-Based Continual Learning," in *Eur. Conf. Comput. Vis.*, vol. 15071, 2024, pp. 163–179.

[46] D. Kim, S. Yoon, D. Park, Y. Lee, H. Song, J. Bang, and J.-G. Lee, "One Size Fits All for Semantic Shifts: Adaptive Prompt Tuning for Continual Learning," in *Int. Conf. Mach. Learn.*, 2024, pp. 24 658–24 673.

[47] Z. Li, L. Zhao, Z. Zhang, H. Zhang, D. Liu, T. Liu, and D. N. Metaxas, "Steering Prototypes with Prompt-Tuning for Rehearsal-Free Continual Learning," in *Winter Conf. Appl. Comput. Vis.*, 2024, pp. 2523–2533.

[48] A. Roy, R. Moulick, V. K. Verma, S. Ghosh, and A. Das, "Convolutional Prompting meets Language Models for Continual Learning," in *Comput. Vis. Pattern Recognit.*, 2024, pp. 23 616–23 626.

[49] Y. Lu, S. Zhang, D. Cheng, Y. Xing, N. Wang, P. Wang, and Y. Zhang, "Visual Prompt Tuning in Null Space for Continual Learning," in *Neural Inf. Process. Syst.*, 2024, pp. 7878–7901.

[50] H. Chen, P. Wang, Z. Zhou, X. Zhang, Z. Wu, and Y.-G. Jiang, "Achieving More with Less: Additive Prompt Tuning for Rehearsal-Free Class-Incremental Learning," in *Int. Conf. Comput. Vis.*, 2025, pp. 340–349.

[51] Y. Hu, Z. Liang, F. Yang, Q. Hou, X. Liu, and M.-M. Cheng, "KAC: Kolmogorov-Arnold Classifier for Continual Learning," in *Comput. Vis. Pattern Recognit.*, 2025, pp. 15 297–15 307.

[52] Y. Lu, J. Tan, S. Zhang, Y. Xing, G. Liang, and Y. Zhang, "Nearest-Neighbor Class Prototype Prompt and Simulated Logits for Continual Learning," *Pattern Recognition*, vol. 169, p. 111933, 2025.

[53] H. Chen, Z. Wu, X. Han, M. Jia, and Y.-G. Jiang, "PromptFusion: Decoupling Stability and Plasticity for Continual Learning," in *Eur. Conf. Comput. Vis.*, vol. 15145, 2024, pp. 196–212.

[54] M. Menabue, E. Frascaroli, M. Boschini, E. Sangineto, L. Bonicelli, A. Porrello, and S. Calderara, "Semantic Residual Prompts for Continual Learning," in *Eur. Conf. Comput. Vis.*, vol. 15119, 2024, pp. 1–18.

[55] L. Yang, H. Zhao, Y. Yu, X. Zeng, and X. Li, "RCS-Prompt: Learning Prompt to Rearrange Class Space for Prompt-Based Continual Learning," in *Eur. Conf. Comput. Vis.*, vol. 15105, 2024, pp. 1–20.

[56] Z. Kang, L. Wang, X. Zhang, and K. Alahari, "Advancing Prompt-Based Methods for Replay-Independent General Continual Learning," in *Int. Conf. Learn. Represent.*, 2025, pp. 88 172–88 190.

[57] J. Qiao, Z. Zhang, X. Tan, C. Chen, Y. Qu, Y. Peng, and Y. Xie, "Prompt Gradient Projection for Continual Learning," in *Int. Conf. Learn. Represent.*, 2024, pp. 3856–3877.

[58] M. Zheng, Y. Tang, Z. Hao, K. Han, Y. Wang, and C. Xu, "Adapt Without Forgetting: Distill Proximity from Dual Teachers in Vision-Language Models," in *Eur. Conf. Comput. Vis.*, vol. 15112, 2024, pp. 109–125.

[59] G. Saha, I. Garg, and K. Roy, "Gradient Projection Memory for Continual Learning," in *Int. Conf. Learn. Represent.*, 2021, pp. 944–961.

[60] S. Wang, X. Li, J. Sun, and Z. Xu, "Training Networks in Null Space of Feature Covariance With Self-Supervision for Incremental Learning," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 47, no. 4, pp. 2563–2580, 2025.

[61] Q. Gao, C. Zhao, Y. Sun, T. Xi, G. Zhang, and B. Ghanem, "A Unified Continual Learning Framework with General Parameter-Efficient Tuning," in *Int. Conf. Comput. Vis.*, 2023, pp. 11 449–11 459.

[62] T. Y. Liu and S. Soatto, "Tangent Model Composition for Ensembling and Continual Fine-tuning," in *Int. Conf. Comput. Vis.*, 2023, pp. 18 630–18 640.

[63] H. Zhuang, Z. Weng, H. Wei, R. Xie, and K.-A. Toh, "ACIL: Analytic Class-Incremental Learning with Absolute Memorization and Privacy Protection," in *Neural Inf. Process. Syst.*, 2022, pp. 11 602–11 614.

[64] H. Zhuang, R. He, K. Tong, Z. Zeng, C. Chen, and Z. Lin, "DS-AL: A Dual-Stream Analytic Learning for Exemplar-Free Class-Incremental Learning," in *AAAI Conf. Artif. Intell.*, vol. 38, Mar. 2024, pp. 17 237–17 244.

[65] H. Zhuang, Y. Chen, D. Fang, R. He, K. Tong, H. Wei, Z. Zeng, and C. Chen, "GACL: Exemplar-Free Generalized Analytic Continual Learning," in *Neural Inf. Process. Syst.*, 2024, pp. 83 024–83 047.

[66] Y. Engel, S. Mannor, and R. Meir, "The Kernel Recursive Least-Squares Algorithm," *IEEE Trans. Signal Process.*, vol. 52, no. 8, pp. 2275–2285, 2004.

[67] G. M. van de Ven, T. Tuytelaars, and A. S. Tolias, "Three types of incremental learning," *Nature Machine Intelligence*, vol. 4, no. 12, pp. 1185–1197, 2022.

[68] L. J. Ba, J. R. Kiros, and G. E. Hinton, "Layer Normalization," *CoRR*, vol. abs/1607.06450, pp. 1–12, 2016.

[69] C. D. Meyer, *Matrix Analysis and Applied Linear Algebra*. Philadelphia, PA, USA: Society for Industrial and Applied Mathematics, 2000.
[70] C. M. Bishop, *Pattern Recognition and Machine Learning*, 1st ed., ser. Information Science and Statistics. New York, NY: Springer, 2006.
[71] D. Hendrycks, S. Basart, N. Mu, S. Kadavath, F. Wang, E. Dorundo, R. Desai, T. Zhu, S. Parajuli, M. Guo, D. Song, J. Steinhardt, and J. Gilmer, "The Many Faces of Robustness: A Critical Analysis of Out-of-Distribution Generalization," in *Int. Conf. Comput. Vis.*, 2021, pp. 8320–8329.
[72] A. Krizhevsky and G. Hinton, "Learning multiple layers of features from tiny images," University of Toronto, Tech. Rep., 2009. [Online]. Available: https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf
[73] X. Peng, Q. Bai, X. Xia, Z. Huang, K. Saenko, and B. Wang, "Moment Matching for Multi-Source Domain Adaptation," in *Int. Conf. Comput. Vis.*, 2019, pp. 1406–1415.
[74] D. Hendrycks, K. Zhao, S. Basart, J. Steinhardt, and D. Song, "Natural Adversarial Examples," in *Comput. Vis. Pattern Recognit.*, 2021, pp. 15 262–15 271.
[75] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. S. Bernstein, A. C. Berg, and L. Fei-Fei, "ImageNet Large Scale Visual Recognition Challenge," *Int. J. Comput. Vis.*, vol. 115, no. 3, pp. 211–252, 2015.
[76] E. D. Cubuk, B. Zoph, D. Mané, V. Vasudevan, and Q. V. Le, "AutoAugment: Learning Augmentation Strategies From Data," in *Comput. Vis. Pattern Recognit.*, 2019, pp. 113–123.
[77] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," in *Int. Conf. Learn. Represent.*, 2015, pp. 1–11.
[78] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Z. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *Neural Inf. Process. Syst.*, 2019, pp. 8024–8035.
[79] R. Wightman, "Pytorch image models," https://github.com/rwightman/pytorch-image-models, 2019.
[80] M. Caron, H. Touvron, I. Misra, H. Jégou, J. Mairal, P. Bojanowski, and A. Joulin, "Emerging Properties in Self-Supervised Vision Transformers," in *Int. Conf. Comput. Vis.*, 2021, pp. 9630–9640.
[81] J. Zhou, C. Wei, H. Wang, W. Shen, C. Xie, A. L. Yuille, and T. Kong, "Image BERT Pre-training with Online Tokenizer," in *Int. Conf. Learn. Represent.*, 2022, pp. 4639–4667.



Yue Lu received the B.E. degree in automation from Beijing Information Science and Technology University, Beijing, China, in 2020, and the M.E. degree in computer science from Northwestern Polytechnical University, Xi'an, China, in 2023. He is currently pursuing the Ph.D degree in computer science with the School of Computer Science, Northwestern Polytechnical University, Xi'an, China. His current research interests include computer vision and deep learning, especially continual learning.



Shizhou Zhang received the B.E. and Ph.D. degrees from Xi'an Jiaotong University, Xi'an, China, in 2010 and 2017, respectively. He is currently an Associate Professor (Tenured) with Northwestern Polytechnical University, Xi'an. His research interests include content-based image analysis, pattern recognition, and machine learning, specifically in vision tasks under continual learning and open world settings, such as image classification, object detection, reidentification.



De Cheng received the B.S. and Ph.D. degrees from Xi'an Jiaotong University, Xi'an, China, in 2011 and 2017, respectively. He is currently an Associate Professor with the School of Telecommunications Engineering, Xidian University, Xi'an. From 2015 to 2017, he was a Visiting Scholar with Carnegie Mellon University, Pittsburgh, PA, USA. His research interests include pattern recognition, machine learning, and multimedia analysis.



Yinghui Xing (Member, IEEE) received the B.S. and Ph.D. degrees from the School of Artificial Intelligence, Xidian University, Xi'an, China, in 2014 and 2020, respectively. She is currently an Associate Professor with the School of Computer Science, Northwestern Polytechnical University, Xi'an. Her research interests include remote sensing image processing, image fusion, and image super resolution.



Guoqiang Liang received the B.S. degree in automation and the Ph.D. degree in pattern recognition and intelligent systems from Xi'an Jiaotong University, Xi'an, China, in 2012 and 2018, respectively. From August 2018 to August 2020, he did his post-doctoral research with the School of Computer Science, Northwestern Polytechnical University (NWPU), Xi'an. Currently, he is an Associate Professor with NWPU. His research interests include pattern recognition and image analysis.



Nannan Wang (Senior Member, IEEE) received the B.Sc. degree in information and computation science from the Xi'an University of Posts and Telecommunications in 2009 and the Ph.D. degree in information and telecommunications engineering from Xidian University in 2015. From September 2011 to September 2013, he was a Visiting Ph.D. Student with the University of Technology Sydney, NSW, Australia. He is currently a Professor with the State Key Laboratory of Integrated Services Networks, Xidian University.



Peng Wang (Member, IEEE) received the bachelor's degree in electrical engineering and automation and the Ph.D. degree in control science and engineering from Beihang University, Beijing, China, in 2004 and 2011, respectively. He is currently a Professor with the School of Computer Science, Northwestern Polytechnical University, Xi'an, China. He was with the School of Computer Science, University of Adelaide, Adelaide, SA, Australia, for approximately four years.



Yanning Zhang (Fellow, IEEE) received the B.E. degree from the Dalian University of Technology, Dalian, China, in 1988, and the Ph.D. degree from the School of Marine Engineering, Northwestern Polytechnical University, Xi'an, China, in 1996. She is currently a Professor and the Executive Dean of the School of Computer Science, Northwestern Polytechnical University. Her research interests include computer vision and pattern recognition, image and video processing, and intelligent information processing.